



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Department of Computer Science and Engineering

CS23334 Fundamentals of Data Science Lab

III semester II Year (2023R)

Name of the Student : ANIRUDH R

Register Number : 240701039

In [2]: Create a pandas DataFrame using the following dictionary:

program: -

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)

print(df)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

In [4]: Read a CSV file named data.csv into a DataFrame and print the first 5 rows.

```
import pandas as pd

csv_data = """Name,Age,City
Alice,25,New York
Bob,30,Los Angeles
Charlie,35,Chicago
David,40,Houston
Eve,28,Seattle
"""

with open('data.csv', 'w') as file:
    file.write(csv_data)

df = pd.read_csv('data.csv')

print(df.head())
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago
3	David	40	Houston
4	Eve	28	Seattle

In [5]: From a DataFrame df, **print** only the rows where Age **is** greater than 28.

```
filtered_df = df[df['Age'] > 28]
print(filtered_df)
```

	Name	Age	City
1	Bob	30	Los Angeles
2	Charlie	35	Chicago
3	David	40	Houston

In []:

In [7]: Add a new column Salary **with** values [50000, 60000, 70000].

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
df['Salary'] = [50000, 60000, 70000]
print(df)
```

	Name	Age	City	Salary
0	Alice	25	New York	50000
1	Bob	30	Los Angeles	60000
2	Charlie	35	Chicago	70000

In [9]: Given a DataFrame with columns Department and Salary, find the average salary per

```
import pandas as pd

data = {
    'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance', 'IT'],
    'Salary': [50000, 70000, 52000, 71000, 60000, 62000, 73000]
}

df = pd.DataFrame(data)

average_salary = df.groupby('Department')['Salary'].mean()

print("Original DataFrame:")
print(df)
print("\nAverage Salary per Department:")
print(average_salary)
```

Original DataFrame:

	Department	Salary
0	HR	50000
1	IT	70000
2	HR	52000
3	IT	71000
4	Finance	60000
5	Finance	62000
6	IT	73000

Average Salary per Department:

Department	
Finance	61000.000000
HR	51000.000000
IT	71333.333333

Name: Salary, dtype: float64

In [10]: Replace all occurrences of 'New York' in City with 'NYC'.

```
import pandas as pd
import sqlite3

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)
df['City'] = df['City'].replace('New York', 'NYC')

conn = sqlite3.connect('sample_database.db')
df.to_sql('people', conn, if_exists='replace', index=False)

result_df = pd.read_sql_query("SELECT * FROM people", conn)
print(result_df)

conn.close()
```

	Name	Age	City
0	Alice	25	NYC
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

In [11]: Q: Drop the Age column from the DataFrame.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)
df = df.drop('Age', axis=1)
print(df)
```

	Name	City
0	Alice	New York
1	Bob	Los Angeles
2	Charlie	Chicago

In [13]: Q: Sort the DataFrame by Salary in descending order.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago'],
    'Salary': [50000, 60000, 70000]
}

df = pd.DataFrame(data)
sorted_df = df.sort_values(by='Salary', ascending=False)
print(sorted_df)
```

	Name	Age	City	Salary
2	Charlie	35	Chicago	70000
1	Bob	30	Los Angeles	60000
0	Alice	25	New York	50000

In [14]: Q: Check for missing values in the DataFrame.

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', None],
    'Age': [25, None, 35, 40],
    'City': ['New York', 'Los Angeles', None, 'Houston'],
    'Salary': [50000, 60000, 70000, None]
}

df = pd.DataFrame(data)
print(df.isnull().sum())
```

```
Name      1
Age        1
City       1
Salary     1
dtype: int64
```

In [19]: Q: Save the DataFrame to a CSV file named output.csv.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago'],
    'Salary': [50000, 60000, 70000]
}

df = pd.DataFrame(data)
df.to_csv('output.csv', index=False)
```

In []:

```
In [1]: import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'Marks': [85, 67, 90, 75],
    'City': ['Delhi', 'Mumbai', 'Delhi', 'Chennai']
}
df = pd.DataFrame(data)

print(df.head(2))
print(df.tail(2))
print(df.shape)
df.info()
print(df.describe())
```

```

      Name  Age  Marks  City
0  Alice   25     85  Delhi
1   Bob   30     67  Mumbai
      Name  Age  Marks  City
2  Charlie  35     90  Delhi
3   David  40     75  Chennai
(4, 4)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Name    4 non-null        object
 1   Age      4 non-null        int64
 2   Marks    4 non-null        int64
 3   City     4 non-null        object
dtypes: int64(2), object(2)
memory usage: 256.0+ bytes
      Age      Marks
count  4.000000  4.000000
mean   32.500000  79.250000
std     6.454972  10.275375
min    25.000000  67.000000
25%    28.750000  73.000000
50%    32.500000  80.000000
75%    36.250000  86.250000
max    40.000000  90.000000
```

```
In [2]: import pandas as pd

data = {
    'Employee': ['John', 'Sara', 'Tim', 'Anna', 'Mike'],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR'],
    'Salary': [50000, 60000, 55000, 62000, 52000],
    'Experience': [5, 3, 8, 6, 4]
}

df_emp = pd.DataFrame(data)

print(df_emp['Department'])
print(df_emp[['Employee', 'Salary']])
print(df_emp.loc[2])
print(df_emp.iloc[4])
print(df_emp.loc[:, 'Experience'])
print(df_emp.loc[1:3, 'Department': 'Salary'])
```

```
0      HR
1      IT
2  Finance
3      IT
4      HR
Name: Department, dtype: object
Employee  Salary
0      John  50000
1      Sara  60000
2      Tim   55000
3      Anna  62000
4      Mike  52000
Employee      Tim
Department  Finance
Salary      55000
Experience      8
Name: 2, dtype: object
Employee      Mike
Department      HR
Salary      52000
Experience      4
Name: 4, dtype: object
0      5
1      3
2      8
3      6
4      4
Name: Experience, dtype: int64
Department  Salary
1      IT   60000
2  Finance   55000
3      IT   62000
```

```
In [4]: import pandas as pd

data = {
    'Student': ['Jathin', 'Anirudh', 'Giri', 'Kathir', 'Goutham'],
    'Score': [78, 92, 85, 87, 90],
    'Age': [19, 18, 19, 19, 18],
    'Area': ['Vysarpadi', 'Mandaveli', 'Mogappair', 'Vadapalani', 'Kilpauk']
}

df_students = pd.DataFrame(data)
filtered_df = df_students[df_students['Score'] > 80]
print(filtered_df)
```

	Student	Score	Age	Area
1	Anirudh	92	18	Mandaveli
2	Giri	85	19	Mogappair
3	Kathir	87	19	Vadapalani
4	Goutham	90	18	Kilpauk

```
In [5]: import pandas as pd

data = {
    'Product': ['Pen', 'Pencil', 'Eraser'],
    'Price': [10, 5, 3],
    'Stock': [100, 200, 150]
}
df = pd.DataFrame(data)

df['Discounted_Price'] = df['Price'] * 0.9
df.loc[1, 'Stock'] = 180
df = df.rename(columns={'Discounted_Price': 'Discount Price'})
df = df.drop('Discount Price', axis=1)
df = df.drop(index=0)

print(df)
```

	Product	Price	Stock
1	Pencil	5	180
2	Eraser	3	150

```
In [6]: import pandas as pd

data = {
    'Region': ['North', 'South', 'North', 'East', 'South', 'East'],
    'Sales': [200, 300, 250, 400, 150, 350]
}
df_sales = pd.DataFrame(data)

print("Total Sales:", df_sales['Sales'].sum())
print("\nAverage Sales per Region:\n", df_sales.groupby('Region')['Sales'].mean())
print("\nAggregations per Region:\n", df_sales.groupby('Region').agg({'Sales': ['
```

Total Sales: 1650

Average Sales per Region:

Region	
East	375.0
North	225.0
South	225.0

Name: Sales, dtype: float64

Aggregations per Region:

	Sales		
	min	max	mean
Region			
East	350	400	375.0
North	200	250	225.0
South	150	300	225.0

```
In [7]: import pandas as pd

df_products = pd.DataFrame({
    'Product': ['Iphone', 'Samsung', 'Oneplus', 'Vivo'],
    'Price': [108000, 79500, 32999, 24999],
    'Stock': [6500, 2000, 1500, 800]
})

print("Sort by Price ascending:\n", df_products.sort_values(by='Price'))
print("\nSort by Stock descending, then Price ascending:\n", df_products.sort_val
```

Sort by Price ascending:

	Product	Price	Stock
3	Vivo	24999	800
2	Oneplus	32999	1500
1	Samsung	79500	2000
0	Iphone	108000	6500

Sort by Stock descending, then Price ascending:

	Product	Price	Stock
0	Iphone	108000	6500
1	Samsung	79500	2000
2	Oneplus	32999	1500
3	Vivo	24999	800

```
In [8]: import pandas as pd
import numpy as np

df_missing = pd.DataFrame({
    'A': [1, 2, None, 4],
    'B': [None, 2, 3, 4]
})

print("Check for nulls:\n", df_missing.isnull())
print("\nDrop rows with nulls:\n", df_missing.dropna())
print("\nForward fill missing values:\n", df_missing.fillna(method='ffill'))
```

Check for nulls:

	A	B
0	False	True
1	False	False
2	True	False
3	False	False

Drop rows with nulls:

	A	B
1	2.0	2.0
3	4.0	4.0

Forward fill missing values:

	A	B
0	1.0	NaN
1	2.0	2.0
2	2.0	3.0
3	4.0	4.0

```
In [9]: import pandas as pd

df_left = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['John', 'Jane', 'Joe']})
df_right = pd.DataFrame({'ID': [2, 3, 4], 'Score': [88, 92, 85]})

print("Inner Merge on ID:\n", pd.merge(df_left, df_right, on='ID'))
print("\nOuter Merge on ID:\n", pd.merge(df_left, df_right, on='ID', how='outer'))
print("\nConcatenate vertically:\n", pd.concat([df_left, df_right], ignore_index=
```

Inner Merge on ID:

	ID	Name	Score
0	2	Jane	88
1	3	Joe	92

Outer Merge on ID:

	ID	Name	Score
0	1	John	NaN
1	2	Jane	88.0
2	3	Joe	92.0
3	4	NaN	85.0

Concatenate vertically:

	ID	Name	Score
0	1	John	NaN
1	2	Jane	NaN
2	3	Joe	NaN
3	2	NaN	88.0
4	3	NaN	92.0
5	4	NaN	85.0

```
In [10]: import pandas as pd

df_example = pd.DataFrame({
    'X': [1, 2, 3],
    'Y': ['a', 'b', 'c']
})

df_example.to_csv('example.csv', index=False)
df_read = pd.read_csv('example.csv')
print("Data read from CSV:\n", df_read)

df_example.to_excel('example.xlsx', index=False)
print("Saved example.xlsx")
```

Data read from CSV:

	X	Y
0	1	a
1	2	b
2	3	c

Saved example.xlsx

```
In [11]: import pandas as pd

df_idx = pd.DataFrame({
    'City': ['Chennai', 'Vellore', 'Madurai'],
    'Population': [8_000_000, 4_000_000, 900_000]
})

df_idx = df_idx.set_index('City')
print("Index labels:\n", df_idx.index)
print("Column labels:\n", df_idx.columns)

df_idx = df_idx.reset_index()
print("\nAfter resetting index:\n", df_idx)
```

Index labels:

Index(['Chennai', 'Vellore', 'Madurai'], dtype='object', name='City')

Column labels:

Index(['Population'], dtype='object')

After resetting index:

	City	Population
0	Chennai	8000000
1	Vellore	4000000
2	Madurai	900000

In []:

```
In [1]: import pandas as pd

data = {
    'Name': ['John Cena', 'Undertaker', 'Roman Reigns', 'Big Show'],
    'Age': [49, 52, 46, 55],
    'City': ['New York', 'Los Angeles', 'United Kingdom', 'California']
}
df = pd.DataFrame(data)

print("First two rows:")
print(df.head(2))

print("\nColumn names:")
print(df.columns.tolist())

print("\nShape of DataFrame:")
print(df.shape)

print("\nSummary info:")
df.info()
```

First two rows:

	Name	Age	City
0	John Cena	49	New York
1	Undertaker	52	Los Angeles

Column names:

['Name', 'Age', 'City']

Shape of DataFrame:

(4, 3)

Summary info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4 entries, 0 to 3

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Name	4 non-null	object
1	Age	4 non-null	int64
2	City	4 non-null	object

dtypes: int64(1), object(2)

memory usage: 224.0+ bytes

```
In [3]: import pandas as pd

data = {
    'Name': ['Shin Chan', 'Neny Chan', 'Masao', 'Kazama'],
    'Age': [9, 10, 9, 9],
    'City': ['America', 'London', 'Kasukabe', 'Russia']
}
df = pd.DataFrame(data)

print("Select Name column:")
print(df['Name'])

print("\nSelect Name and City columns:")
print(df[['Name', 'City']])

print("\nSelect second row using iloc:")
print(df.iloc[1])

print("\nSelect row where Name is 'Charlie' using loc:")
print(df.loc[df['Name'] == 'Charlie'])
```

Select Name column:

```
0    Shin Chan
1    Neny Chan
2         Masao
3     Kazama
```

Name: Name, dtype: object

Select Name and City columns:

```
      Name      City
0  Shin Chan  America
1  Neny Chan   London
2     Masao  Kasukabe
3    Kazama   Russia
```

Select second row using iloc:

```
Name    Neny Chan
Age         10
City      London
```

Name: 1, dtype: object

Select row where Name is 'Charlie' using loc:

Empty DataFrame

Columns: [Name, Age, City]

Index: []

```
In [6]: import pandas as pd

data = {
    'Name': ['Ajith', 'Vijay', 'Surya', 'Dhanush'],
    'Age': [50, 48, 41, 42],
    'City': ['Chennai', 'Madurai', 'Salem', 'VadaChennai']
}
df = pd.DataFrame(data)

print("People older than 25:")
print(df[df['Age'] > 25])

print("\nPeople living in Chicago or Houston:")
print(df[df['City'].isin(['Chicago', 'Houston'])])

print("\nPeople whose age is between 23 and 30:")
print(df[(df['Age'] >= 23) & (df['Age'] <= 30)])
```

People older than 25:

	Name	Age	City
0	Ajith	50	Chennai
1	Vijay	48	Madurai
2	Surya	41	Salem
3	Dhanush	42	VadaChennai

People living in Chicago or Houston:

Empty DataFrame

Columns: [Name, Age, City]

Index: []

People whose age is between 23 and 30:

Empty DataFrame

Columns: [Name, Age, City]

Index: []

```
In [7]: import pandas as pd

data = {
    'Name': ['Nobita', 'Gian', 'Sunio', 'Suzuka'],
    'Age': [24, 27, 22, 32],
    'City': ['China', 'London', 'Hong Kong', 'Japan']
}
df = pd.DataFrame(data)
df['Score'] = [85, 90, 88, 95]
print("Added Score column:")
print(df)
df.loc[df['Name'] == 'Bob', 'Age'] = 28
print("\nChanged Bob's age to 28:")
print(df)
df = df.drop('City', axis=1)
print("\nRemoved City column:")
print(df)
df = df[df['Name'] != 'David']
print("\nDropped David's row:")
print(df)
```

Added Score column:

	Name	Age	City	Score
0	Nobita	24	China	85
1	Gian	27	London	90
2	Sunio	22	Hong Kong	88
3	Suzuka	32	Japan	95

Changed Bob's age to 28:

	Name	Age	City	Score
0	Nobita	24	China	85
1	Gian	27	London	90
2	Sunio	22	Hong Kong	88
3	Suzuka	32	Japan	95

Removed City column:

	Name	Age	Score
0	Nobita	24	85
1	Gian	27	90
2	Sunio	22	88
3	Suzuka	32	95

Dropped David's row:

	Name	Age	Score
0	Nobita	24	85
1	Gian	27	90
2	Sunio	22	88
3	Suzuka	32	95

```
In [8]: import pandas as pd

data = {
    'Department': ['HR', 'IT', 'HR', 'IT'],
    'Salary': [30000, 50000, 35000, 55000],
    'Experience': [2, 5, 3, 6]
}
df = pd.DataFrame(data)

print("Average Salary by Department:")
print(df.groupby('Department')['Salary'].mean())

print("\nMaximum Experience by Department:")
print(df.groupby('Department')['Experience'].max())

print("\nTotal Salary paid:")
print(df['Salary'].sum())
```

Average Salary by Department:

Department

HR 32500.0

IT 52500.0

Name: Salary, dtype: float64

Maximum Experience by Department:

Department

HR 3

IT 6

Name: Experience, dtype: int64

Total Salary paid:

170000

```
In [10]: import pandas as pd

data = {
    'Student': ['Rohit Sharma', 'Virat Kohli', 'M S Dhoni', 'Hardik Pandya'],
    'Marks': [80, None, 75, 90]
}
df = pd.DataFrame(data)

print("Fill missing marks with 0:")
print(df.fillna(0))

print("\nDrop rows with missing values:")
print(df.dropna())

print("\nSort by Marks descending:")
print(df.sort_values(by='Marks', ascending=False))
```

Fill missing marks with 0:

	Student	Marks
0	Rohit Sharma	80.0
1	Virat Kohli	0.0
2	M S Dhoni	75.0
3	Hardik Pandya	90.0

Drop rows with missing values:

	Student	Marks
0	Rohit Sharma	80.0
2	M S Dhoni	75.0
3	Hardik Pandya	90.0

Sort by Marks descending:

	Student	Marks
3	Hardik Pandya	90.0
0	Rohit Sharma	80.0
2	M S Dhoni	75.0
1	Virat Kohli	NaN

```
In [14]: import pandas as pd

data = {
    'Name': ['Smithi Mandana', 'Harmanpreet Kaur', 'Radha Yadav', 'Ellis Perry'],
    'Age': [29, 28, 24, 32],
    'City': ['India', 'Mumbai', 'California', 'Bangalore']
}
df = pd.DataFrame(data)
df.to_csv('students.csv', index=False)
print("Saved DataFrame to students.csv")

employees_data = {
    'Name': ['Tom', 'Jerry', 'Spike'],
    'Age': [29, 31, 28],
    'Department': ['Sales', 'HR', 'Marketing']
}
df_employees = pd.DataFrame(employees_data)
df_employees.to_csv('employees.csv', index=False)

df_loaded = pd.read_csv('employees.csv')
print("\nLoaded employees.csv:")
print(df_loaded)
df_indexed = df_loaded.set_index('Name')
print("\nSet Name as index:")
print(df_indexed)
df_reset = df_indexed.reset_index()
print("\nReset index:")
print(df_reset)
```

Saved DataFrame to students.csv

Loaded employees.csv:

	Name	Age	Department
0	Tom	29	Sales
1	Jerry	31	HR
2	Spike	28	Marketing

Set Name as index:

	Age	Department
Name		
Tom	29	Sales
Jerry	31	HR
Spike	28	Marketing

Reset index:

	Name	Age	Department
0	Tom	29	Sales
1	Jerry	31	HR
2	Spike	28	Marketing

```
In [15]: import pandas as pd

data = {
    'Product': ['Laptop', 'Tablet', 'Smartphone', 'Monitor', 'Keyboard'],
    'Price': [70000, 30000, 25000, 15000, 2000],
    'Stock': [10, 25, 50, 15, 100]
}
df = pd.DataFrame(data)

print(df.loc[[0, 2]])
```

	Product	Price	Stock
0	Laptop	70000	10
2	Smartphone	25000	50

```
In [16]: import pandas as pd

data = {
    'Subject': ['Math', 'Science', 'English'],
    'Marks': [88, 92, 85]
}
df = pd.DataFrame(data, index=['Student1', 'Student2', 'Student3'])

print(df.loc['Student2'])
```

Subject	Science
Marks	92

Name: Student2, dtype: object

```
In [ ]:
```

```

In [1]: import pandas as pd
cricketer_data = {
    'Name': [
        'Virat Kohli', 'Rohit Sharma', 'MS Dhoni', 'Jasprit Bumrah', 'Ravindra
        'Steve Smith', 'Joe Root', 'Kane Williamson', 'Ben Stokes', 'David Warr
        'Babar Azam', 'Shaheen Afridi', 'Quinton de Kock', 'Faf du Plessis', 'F
        'Shakib Al Hasan', 'Chris Gayle', 'AB de Villiers', 'Trent Boult', 'Pat
        'Jonny Bairstow', 'Jofra Archer', 'Kagiso Rabada', 'Imam-ul-Haq', 'Jos
    ],
    'Age': [
        36, 38, 44, 31, 36, 36, 34, 34, 34, 38, 30, 25, 32, 41, 26, 38, 45, 41,
    ],
    'Country': [
        'India', 'India', 'India', 'India', 'India',
        'Australia', 'England', 'New Zealand', 'England', 'Australia',
        'Pakistan', 'Pakistan', 'South Africa', 'South Africa', 'Afghanistan',
        'Bangladesh', 'West Indies', 'South Africa', 'New Zealand', 'Australia'
        'England', 'England', 'South Africa', 'Pakistan', 'England'
    ],
    'Matches_Played': [
        520, 480, 538, 250, 450, 400, 410, 380, 395, 420, 280, 150, 350, 450, 2
    ],
    'Runs': [
        26000, 22000, 17266, 150, 7500, 19000, 18500, 17500, 15000, 20500, 1200
    ],
    'Wickets': [
        4, 8, 2, 450, 550, 15, 20, 5, 200, 5, 2, 350, 1, 10, 300, 600, 25, 10,
    ],
    'Batting_Avg': [
        54.2, 48.5, 44.3, 10.5, 34.5, 52.1, 50.2, 49.8, 38.0, 46.5, 51.0, 15.0,
    ],
    'Bowling_Avg': [
        80.5, 75.0, 90.0, 22.0, 29.0, 65.0, 55.0, 85.0, 32.0, 70.0, 95.0, 25.0,
    ],
    'Role': [
        'Batsman', 'Batsman', 'Wicketkeeper-Batsman', 'Bowler', 'All-rounder',
        'Batsman', 'Batsman', 'Batsman', 'All-rounder', 'Batsman',
        'Batsman', 'Bowler', 'Wicketkeeper-Batsman', 'Batsman', 'Bowler',
        'All-rounder', 'Batsman', 'Batsman', 'Bowler', 'Bowler',
        'Wicketkeeper-Batsman', 'Bowler', 'Bowler', 'Batsman', 'Wicketkeeper-Ba
    ]
}

df = pd.DataFrame(cricketer_data)
df.to_csv('cricketer_stats.csv', index=False)
print("CSV file 'cricketer_stats.csv' created successfully.")
print(df)

```

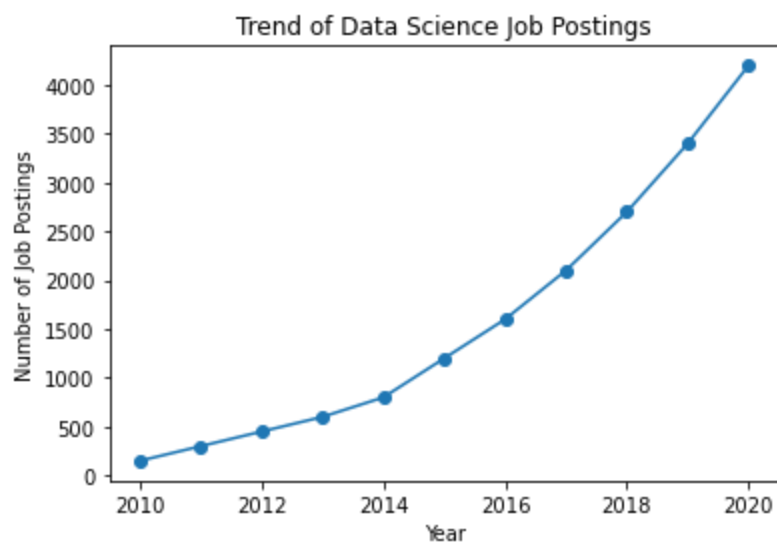
CSV file 'cricketer_stats.csv' created successfully.

	Name	Age	Country	Matches_Played	Runs	Wickets	\
0	Virat Kohli	36	India	520	26000	4	
1	Rohit Sharma	38	India	480	22000	8	
2	MS Dhoni	44	India	538	17266	2	
3	Jasprit Bumrah	31	India	250	150	450	
4	Ravindra Jadeja	36	India	450	7500	550	
5	Steve Smith	36	Australia	400	19000	15	
6	Joe Root	34	England	410	18500	20	
7	Kane Williamson	34	New Zealand	380	17500	5	
8	Ben Stokes	34	England	395	15000	200	
9	David Warner	38	Australia	420	20500	5	
10	Babar Azam	30	Pakistan	280	12000	2	
11	Shaheen Afridi	25	Pakistan	150	300	350	
12	Quinton de Kock	32	South Africa	350	15500	1	
13	Faf du Plessis	41	South Africa	450	18000	10	
14	Rashid Khan	26	Afghanistan	200	500	300	
15	Shakib Al Hasan	38	Bangladesh	490	15000	600	
16	Chris Gayle	45	West Indies	500	21000	25	
17	AB de Villiers	41	South Africa	450	20000	10	
18	Trent Boult	35	New Zealand	280	800	400	
19	Pat Cummins	32	Australia	300	950	350	
20	Jonny Bairstow	35	England	380	16000	10	
21	Jofra Archer	30	England	180	250	250	
22	Kagiso Rabada	29	South Africa	250	500	320	
23	Imam-ul-Haq	29	Pakistan	210	9000	1	
24	Jos Buttler	34	England	350	15000	5	

	Batting_Avg	Bowling_Avg	Role
0	54.2	80.5	Batsman
1	48.5	75.0	Batsman
2	44.3	90.0	Wicketkeeper-Batsman
3	10.5	22.0	Bowler
4	34.5	29.0	All-rounder
5	52.1	65.0	Batsman
6	50.2	55.0	Batsman
7	49.8	85.0	Batsman
8	38.0	32.0	All-rounder
9	46.5	70.0	Batsman
10	51.0	95.0	Batsman
11	15.0	25.0	Bowler
12	42.0	120.0	Wicketkeeper-Batsman
13	45.0	88.0	Batsman
14	18.0	20.0	Bowler
15	35.5	28.0	All-rounder
16	40.0	50.0	Batsman
17	48.0	60.0	Batsman
18	25.0	26.0	Bowler
19	28.0	27.0	Bowler
20	45.0	68.0	Wicketkeeper-Batsman
21	12.0	30.0	Bowler
22	22.0	24.0	Bowler
23	45.0	110.0	Batsman
24	42.5	95.0	Wicketkeeper-Batsman

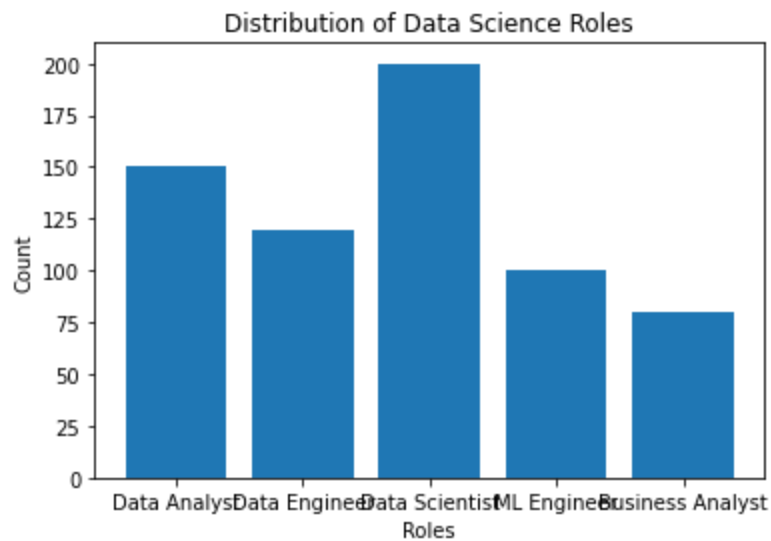
```
In [3]: import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Year': list(range(2010, 2021)),
    'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]
}
df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```



```
In [4]: import matplotlib.pyplot as plt

roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer', 'Business Analyst']
counts = [150, 120, 200, 100, 80]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Roles')
plt.ylabel('Count')
plt.show()
```



```
In [5]: import pandas as pd

structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})
unstructured_data = "This is an unstructured data example: free-flowing text with"
semi_structured_data = {
    "Name": "David",
    "Skills": ["Python", "Machine Learning", "Data Analysis"],
    "Experience": {"Years": 5, "Domain": "AI"}
}
print("Structured Data (DataFrame):\n", structured_data, "\n")
print("Unstructured Data (Text):\n", unstructured_data, "\n")
print("Semi-structured Data (Dictionary):\n", semi_structured_data)
```

Structured Data (DataFrame):

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Charlie	35

Unstructured Data (Text):

This is an unstructured data example: free-flowing text without fixed format.

Semi-structured Data (Dictionary):

```
{'Name': 'David', 'Skills': ['Python', 'Machine Learning', 'Data Analysis'],
'Experience': {'Years': 5, 'Domain': 'AI'}}
```

```
In [8]: from cryptography.fernet import Fernet
```

```
key = Fernet.generate_key()
cipher1 = Fernet(key)

original_text = "Rajalakshmi Engineering College"
cipher_text = cipher1.encrypt(original_text.encode())
print("Encrypted Data:", cipher_text)

cipher2 = Fernet(key)
decrypted_text = cipher2.decrypt(cipher_text).decode()

print("Original Data:", original_text)
print("Decrypted Data:", decrypted_text)
```

```
Encrypted Data: b'gAAAAABotl6qRCjrdLrS7ALaVnmIZe2zUsQ6lV00nbPBFKAL91ZaEYrEj1NN
ohAnDfOT3tLOqlXMzIixbUxsC2kx7VsA_QvV6UoHg4zeUFFxQQUe4JVUMY='
Original Data: Rajalakshmi Engineering College
Decrypted Data: Rajalakshmi Engineering College
```

```
In [ ]:
```



```

In [2]: import pandas as pd
import matplotlib.pyplot as plt
import os, re

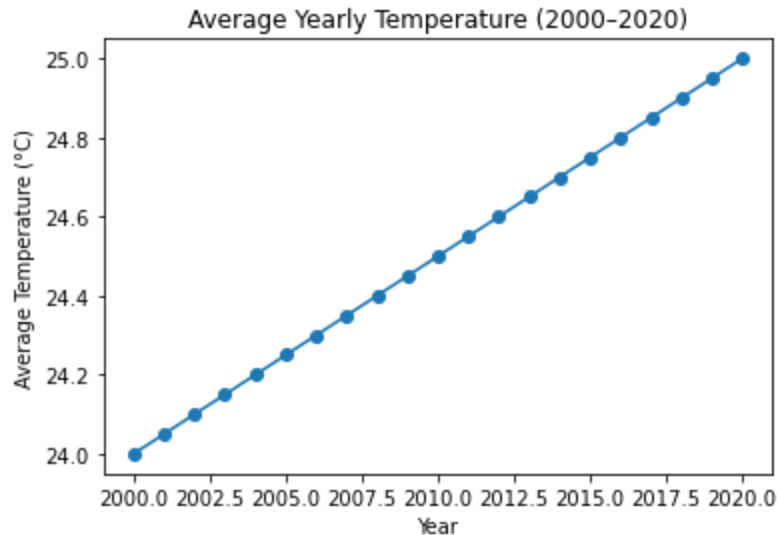
file = "weather.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Year": list(range(2000, 2021)), "AvgTemperature": [24.0
    df0.to_csv(file, index=False)

df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
year_col = next((c for c in df.columns if c == "year" or "year" in c), None)
temp_col = next((c for c in df.columns if c in {"avgtemperature", "averagetemperature"}), None)
if year_col is None or temp_col is None:
    raise ValueError(f"Expected columns like 'Year' and 'AvgTemperature'. Found: {year_col}, {temp_col}")

df[year_col] = pd.to_numeric(df[year_col], errors="coerce")
df[temp_col] = pd.to_numeric(df[temp_col].astype(str).str.extract(r"([+-]?\d*\.\d*)"), errors="coerce")
df = df.dropna(subset=[year_col, temp_col]).sort_values(year_col)

plt.plot(df[year_col], df[temp_col], marker="o")
plt.title("Average Yearly Temperature (2000-2020)")
plt.xlabel("Year")
plt.ylabel("Average Temperature (°C)")
plt.show()

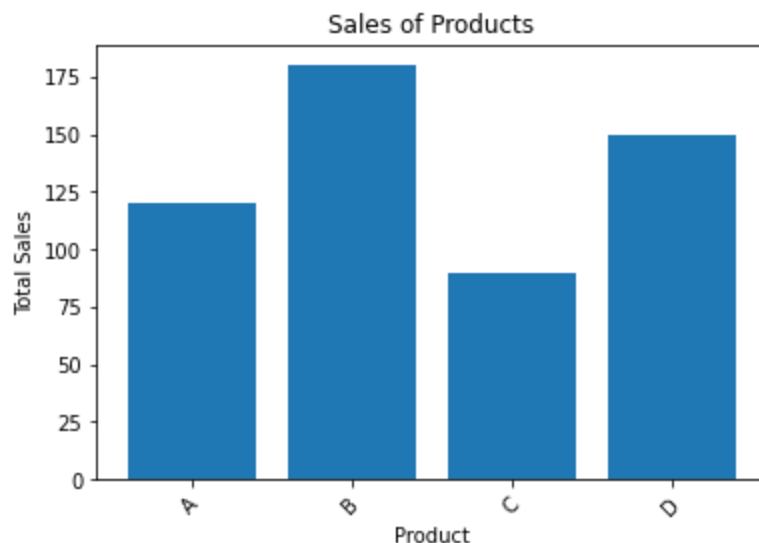
```



```
In [4]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "sales.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Product": ["A", "B", "C", "D"], "Sales": [120, 180, 90, 150]})
    df0.to_csv(file, index=False)

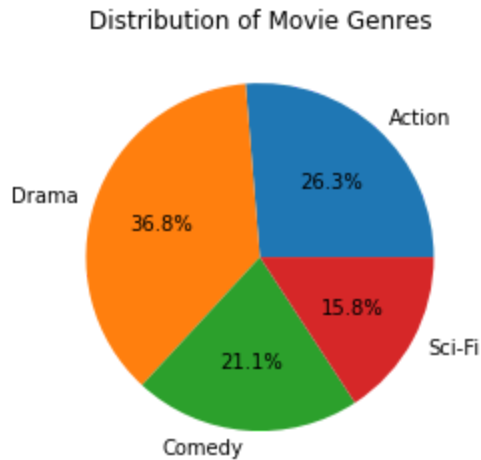
df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
pcol = next((c for c in df.columns if "product" in c), None)
scol = next((c for c in df.columns if "sale" in c), None)
plt.bar(df[pcol], pd.to_numeric(df[scol], errors="coerce"))
plt.title("Sales of Products")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.show()
```



```
In [5]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "movies.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Genre": ["Action", "Drama", "Comedy", "Sci-Fi"], "Count": [
    df0.to_csv(file, index=False)

df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
gcol = next((c for c in df.columns if "genre" in c), None)
ccol = next((c for c in df.columns if "count" in c), None)
plt.pie(pd.to_numeric(df[ccol], errors="coerce"), labels=df[gcol], autopct="%1.1f%%")
plt.title("Distribution of Movie Genres")
plt.show()
```



```

In [8]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "students.csv"
if not os.path.exists(file):
    pd.DataFrame({"MathMarks": [60, 70, 80, 90, 100], "ScienceMarks": [65, 75, 78, 85, 95]})

df = pd.read_csv(file)
orig_cols = list(df.columns)
clean = [re.sub(r"\s+", "", c).strip().lower() for c in orig_cols]
df.columns = clean

mcol = next((c for c in df.columns if any(k in c for k in ("math", "mathematics"))))
scol = next((c for c in df.columns if any(k in c for k in ("science", "sci"))), None)

if mcol is None or scol is None:
    numcols = df.select_dtypes(include=["number"]).columns.tolist()
    if len(numcols) >= 2:
        if mcol is None: mcol = numcols[0]
        if scol is None: scol = numcols[1] if numcols[1] != mcol else (numcols[0] if len(numcols) > 2 else numcols[0])

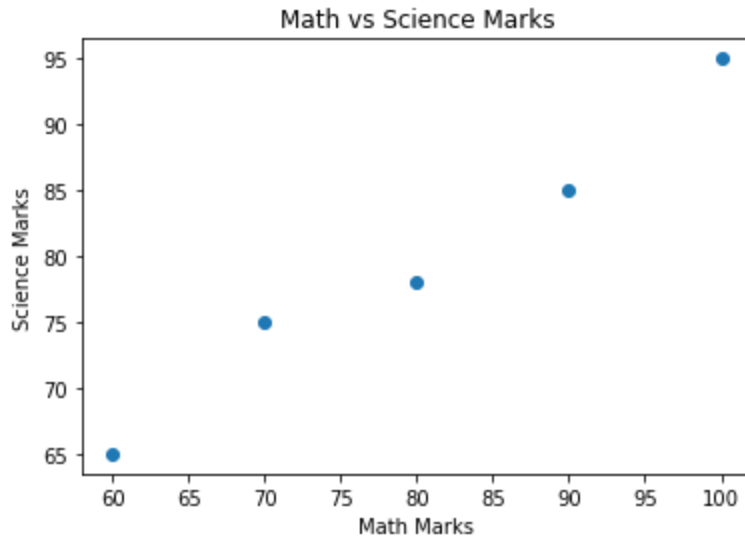
if mcol is None or scol is None:
    df0 = pd.DataFrame({"MathMarks": [60, 70, 80, 90, 100], "ScienceMarks": [65, 75, 78, 85, 95]})
    df0.to_csv(file, index=False)
    df = df0.copy()
    df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
    mcol = next((c for c in df.columns if "math" in c), df.columns[0])
    scol = next((c for c in df.columns if "science" in c), df.columns[1] if len(df.columns) > 1 else df.columns[0])

x = pd.to_numeric(df[mcol], errors="coerce")
y = pd.to_numeric(df[scol], errors="coerce")
df2 = pd.concat([x, y], axis=1).dropna()

if df2.empty:
    df0 = pd.DataFrame({"MathMarks": [60, 70, 80, 90, 100], "ScienceMarks": [65, 75, 78, 85, 95]})
    df0.to_csv(file, index=False)
    x = df0["MathMarks"]
    y = df0["ScienceMarks"]
else:
    x = df2.iloc[:, 0]
    y = df2.iloc[:, 1]

plt.scatter(x, y)
plt.title("Math vs Science Marks")
plt.xlabel("Math Marks")
plt.ylabel("Science Marks")
plt.show()

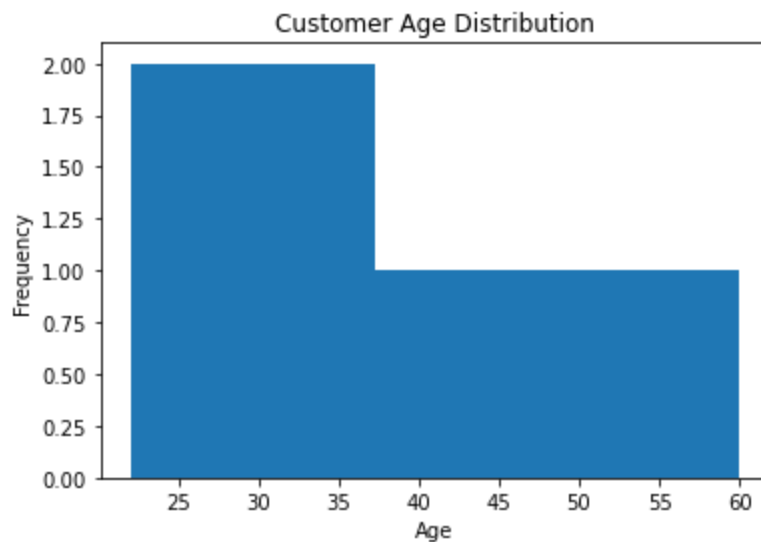
```



```
In [7]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "customers.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Age": [22, 25, 30, 35, 40, 45, 50, 28, 33, 29, 60, 55, 42, 37]})
    df0.to_csv(file, index=False)

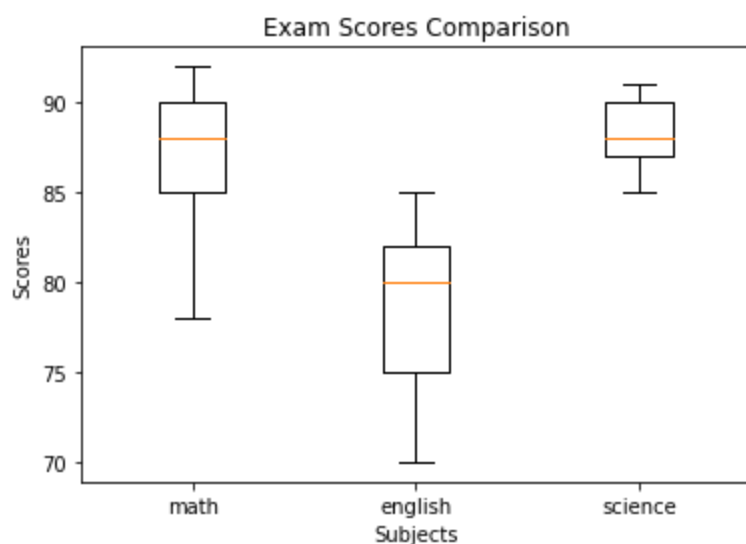
df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
acol = next((c for c in df.columns if "age" in c), None)
plt.hist(pd.to_numeric(df[acol], errors="coerce"), bins=10)
plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



```
In [9]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "exam_scores.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Math": [78, 85, 90, 88, 92], "English": [70, 75, 80, 82, 85], "Science": [85, 88, 90, 92, 95]})
    df0.to_csv(file, index=False)

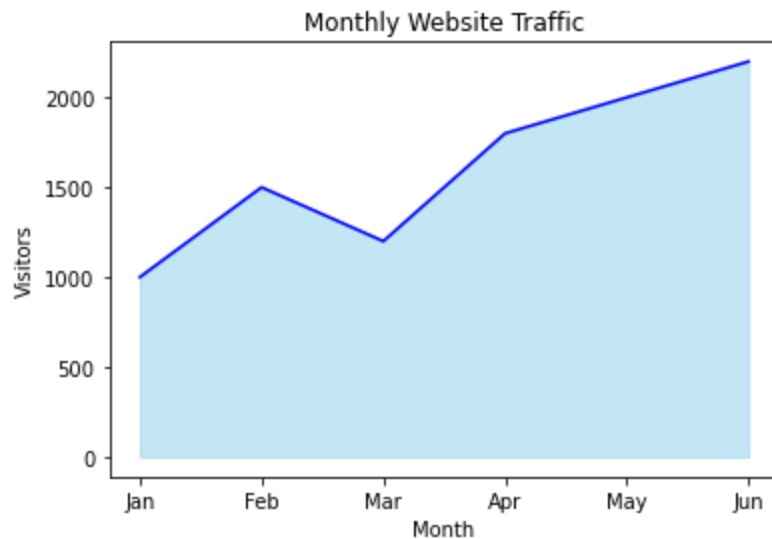
df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
cols = [c for c in df.columns if any(x in c for x in ["math", "english", "science"])]
plt.boxplot([pd.to_numeric(df[c], errors="coerce") for c in cols], labels=cols)
plt.title("Exam Scores Comparison")
plt.xlabel("Subjects")
plt.ylabel("Scores")
plt.show()
```



```
In [10]: import pandas as pd, matplotlib.pyplot as plt, os, re

file = "traffic.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({"Month":["Jan", "Feb", "Mar", "Apr", "May", "Jun"], "Visitors":
    df0.to_csv(file, index=False)

df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
mcol = next((c for c in df.columns if "month" in c), None)
vcol = next((c for c in df.columns if "visitor" in c), None)
plt.fill_between(df[mcol], pd.to_numeric(df[vcol], errors="coerce"), color="skyblue")
plt.plot(df[mcol], pd.to_numeric(df[vcol], errors="coerce"), color="blue")
plt.title("Monthly Website Traffic")
plt.xlabel("Month")
plt.ylabel("Visitors")
plt.show()
```



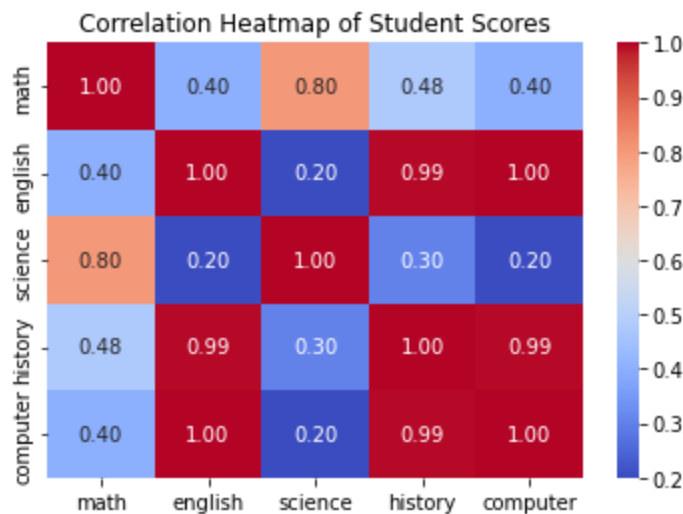
```

In [17]: import pandas as pd, seaborn as sns, matplotlib.pyplot as plt, os, re

file = "students_scores.csv"
if not os.path.exists(file):
    df0 = pd.DataFrame({
        "Math": [80, 85, 90, 95],
        "English": [75, 80, 70, 85],
        "Science": [88, 86, 90, 92],
        "History": [70, 72, 68, 75],
        "Computer": [95, 96, 94, 97]
    })
    df0.to_csv(file, index=False)

df = pd.read_csv(file)
df.columns = [re.sub(r"\s+", "", c).strip().lower() for c in df.columns]
df = df.apply(pd.to_numeric, errors="coerce")
if df.dropna(axis=1, how="all").empty:
    df = pd.DataFrame({
        "math": [80, 85, 90, 95],
        "english": [75, 80, 70, 85],
        "science": [88, 86, 90, 92],
        "history": [70, 72, 68, 75],
        "computer": [95, 96, 94, 97]
    })
corr = df.select_dtypes(include=["number"]).corr()
plt.figure(figsize=(6, 4))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap of Student Scores")
plt.show()

```



In []:


```

In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style='whitegrid')
plt.rcParams["figure.figsize"] = (10, 6)

def create_sample_csv():
    np.random.seed(42)
    dates = pd.date_range(start="2025-01-01", periods=100)
    products = ['Product A', 'Product B', 'Product C']
    regions = ['North', 'South', 'East', 'West']

    data = {
        'Date': np.random.choice(dates, 300),
        'Product': np.random.choice(products, 300),
        'Quantity': np.random.randint(5, 25, size=300),
        'Sales': np.random.uniform(50, 250, size=300),
        'Region': np.random.choice(regions, 300)
    }

    df = pd.DataFrame(data)

    # Introduce some missing values for realism
    for _ in range(5):
        df.loc[np.random.randint(0, 300), 'Sales'] = np.nan
        df.loc[np.random.randint(0, 300), 'Product'] = np.nan
        df.loc[np.random.randint(0, 300), 'Region'] = np.nan

    df.to_csv('sales_data.csv', index=False)

def load_and_clean_data(filepath):
    df = pd.read_csv(filepath)
    df['Sales'].fillna(df['Sales'].mean(), inplace=True)
    df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
    df['Date'] = pd.to_datetime(df['Date'])
    return df

def summary_statistics(df):
    print("\n Summary Statistics:\n")
    print(df.describe())

def sales_by_product(df):
    grouped = df.groupby('Product').agg({'Sales': 'sum', 'Quantity': 'sum'})
    sns.barplot(data=grouped.sort_values('Sales', ascending=False), y='Product')
    plt.title("Total Sales by Product")
    plt.xlabel("Sales")
    plt.ylabel("Product")
    plt.tight_layout()
    plt.show()
    return grouped

def sales_over_time(df):
    time_data = df.groupby('Date')['Sales'].sum().reset_index()
    sns.lineplot(data=time_data, x='Date', y='Sales', color='teal')
    plt.title("Total Sales Over Time")

```

```

plt.xlabel("Date")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()

def region_product_pivot(df):
    pivot = pd.pivot_table(df, values='Sales', index='Region', columns='Product')
    print("📊 Sales Pivot Table (Region vs Product):\n")
    print(pivot)
    return pivot

def correlation_heatmap(df):
    corr = df[['Quantity', 'Sales']].corr()
    sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
    plt.title("Correlation Matrix")
    plt.tight_layout()
    plt.show()

def run_eda(file_path):
    df = load_and_clean_data(file_path)
    print("🔍 First 5 Records:\n")
    print(df.head(), "\n")
    summary_statistics(df)
    sales_by_product(df)
    sales_over_time(df)
    region_product_pivot(df)
    correlation_heatmap(df)

create_sample_csv()
run_eda('sales_data.csv')

```

🔍 First 5 Records:

	Date	Product	Quantity	Sales	Region
0	2025-02-21	Product C	23	197.364495	North
1	2025-04-03	Product B	21	161.548453	East
3	2025-03-13	Product C	16	162.891709	North
4	2025-03-02	Product A	13	125.754525	North
5	2025-01-21	Product B	23	117.489367	South

📊 Summary Statistics:

	Quantity	Sales
count	290.000000	290.000000
mean	14.675862	155.159966
std	5.796291	54.716370
min	5.000000	50.542180
25%	9.000000	112.530805
50%	15.000000	156.949262
75%	20.000000	200.385564
max	34.000000	240.538533

```
In [4]: import matplotlib.pyplot as plt
import numpy as np

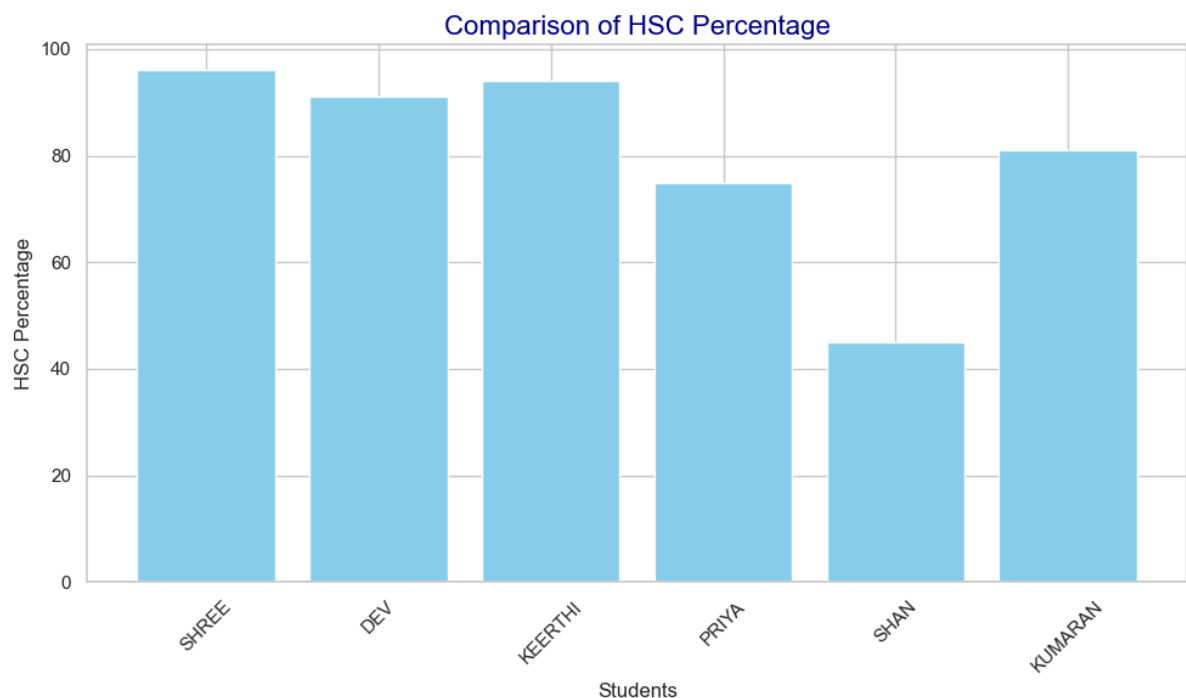
students = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
percentages = np.array([96, 91, 94, 75, 45, 81])

x_pos = np.arange(len(students))

plt.bar(x_pos, percentages, color='skyblue')
plt.xlabel('Students')
plt.ylabel('HSC Percentage')
plt.xticks(x_pos, students, rotation=45)
plt.title('Comparison of HSC Percentage', fontsize=16, color='darkblue')
plt.tight_layout()
plt.show()

highest = students[np.argmax(percentages)]
lowest = students[np.argmin(percentages)]

print(f"Highest Percentage: {highest} with {percentages.max()}%")
print(f"Lowest Percentage: {lowest} with {percentages.min()}%")
```

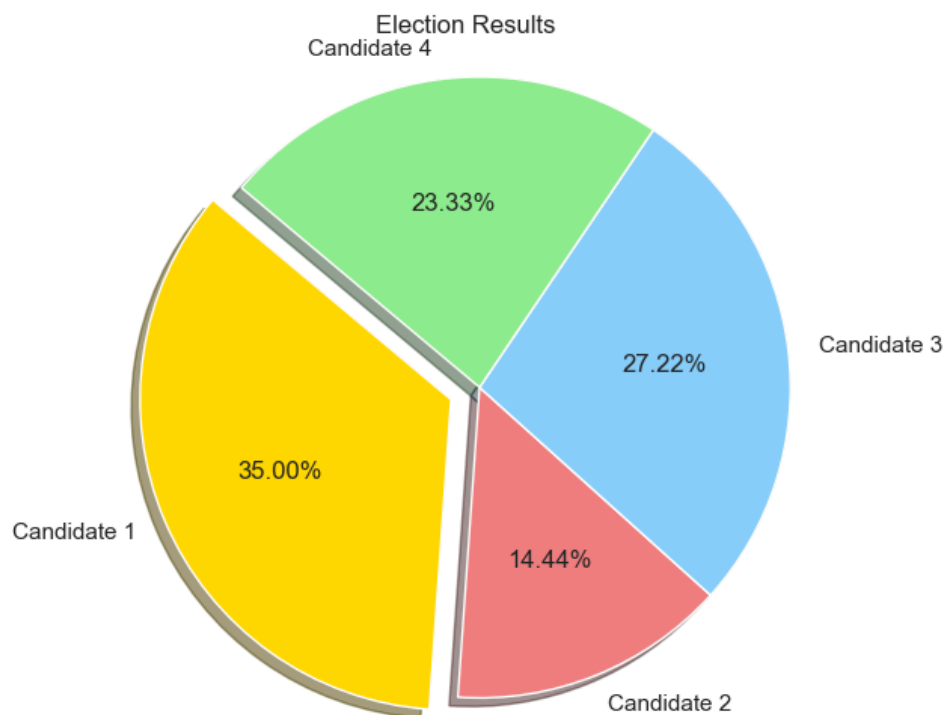


Highest Percentage: SHREE with 96%
Lowest Percentage: SHAN with 45%

```
In [6]: import matplotlib.pyplot as plt

candidates = ['Candidate 1', 'Candidate 2', 'Candidate 3', 'Candidate 4']
votes = [315, 130, 245, 210]
colors = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen']
explode = (0.1, 0, 0, 0) # Highlight Candidate 1

plt.pie(votes, labels=candidates, colors=colors, explode=explode, autopct='%1.1f%%',
plt.title('Election Results')
plt.axis('equal') # Equal aspect ratio ensures the pie chart is circular
plt.show()
```



```
In [11]: from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
from collections import Counter

sample = gutenberg.raw("austen-emma.txt")
tokens = word_tokenize(sample)
wlist = tokens[:50]
freq_counter = Counter(wlist)
print(list(freq_counter.items()))
```

```
[(['', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1),
(''], 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('Woodhouse', 1), ('',
5), ('handsome', 1), ('clever', 1), ('and', 3), ('rich', 1), ('with', 2),
('a', 1), ('comfortable', 1), ('home', 1), ('happy', 1), ('disposition', 1),
('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2),
('best', 1), ('blessings', 1), ('existence', 1), (';', 1), ('had', 1), ('live
d', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('world',
1)]
```

1) Write a Python program to collect, load, and perform initial exploration of the Diabetes dataset using Pandas. Display the first few records of the dataset and summarize its structure and contents.

Exp No: 5 - Data Collection and Initial Exploration

Objective: To collect, load, and perform initial exploration of the diabetes dataset


```
In [6]: # Exp No: 5 - Data Collection and Initial Exploration
# Objective: To collect, load, and perform initial exploration of the diabetes

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Create the diabetes.csv file manually
data = {
    "Pregnancies": [6,1,8,1,0,5,3,10,2,8],
    "Glucose": [148,85,183,89,137,116,78,115,197,125],
    "BloodPressure": [72,66,64,66,40,74,50,0,70,96],
    "SkinThickness": [35,29,0,23,35,0,32,0,45,0],
    "Insulin": [0,0,0,94,168,0,88,0,543,0],
    "BMI": [33.6,26.6,23.3,28.1,43.1,25.6,31.0,35.3,30.5,0.0],
    "DiabetesPedigreeFunction": [0.627,0.351,0.672,0.167,2.288,0.201,0.248,0.133,0.191,0.168],
    "Age": [50,31,32,21,33,30,26,29,53,54],
    "Outcome": [1,0,1,0,1,0,1,0,1,1]
}

df = pd.DataFrame(data)
df.to_csv("diabetes.csv", index=False)
print("diabetes.csv file created successfully!")

# Step 2: Load the dataset from the created CSV file
diabetes_df = pd.read_csv("diabetes.csv")

# Step 3: Display the first few rows
print("\nFirst 5 rows of the dataset:")
print(diabetes_df.head())

# Step 4: Display basic information about the dataset
print("\nDataset Information:")
print(diabetes_df.info())

# Step 5: Display statistical summary
print("\nStatistical Summary:")
print(diabetes_df.describe())

# Step 6: Check for missing values
print("\nMissing Values in Each Column:")
print(diabetes_df.isnull().sum())

# Step 7: Visualize the distribution of the target variable (Outcome)
plt.figure(figsize=(6,4))
sns.countplot(x='Outcome', data=diabetes_df)
plt.title('Distribution of Diabetes Outcome')
plt.xlabel('Outcome (0 = Non-Diabetic, 1 = Diabetic)')
plt.ylabel('Count')
plt.show()

# Step 8: Display correlation heatmap
plt.figure(figsize=(10,8))
sns.heatmap(diabetes_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap of Features')
plt.show()
```


diabetes.csv file created successfully!

First 5 rows of the dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 10 entries, 0 to 9

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	10 non-null	int64
1	Glucose	10 non-null	int64
2	BloodPressure	10 non-null	int64
3	SkinThickness	10 non-null	int64
4	Insulin	10 non-null	int64
5	BMI	10 non-null	float64
6	DiabetesPedigreeFunction	10 non-null	float64
7	Age	10 non-null	int64
8	Outcome	10 non-null	int64

dtypes: float64(2), int64(7)

memory usage: 848.0 bytes

None

Statistical Summary:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	\
count	10.00000	10.000000	10.000000	10.000000	10.000000	
mean	4.40000	127.300000	59.800000	19.900000	89.300000	
std	3.50238	40.075068	25.654976	17.978073	169.937276	
min	0.00000	78.000000	0.000000	0.000000	0.000000	
25%	1.25000	95.500000	53.500000	0.000000	0.000000	
50%	4.00000	120.500000	66.000000	26.000000	0.000000	
75%	7.50000	145.250000	71.500000	34.250000	92.500000	
max	10.00000	197.000000	96.000000	45.000000	543.000000	

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	10.000000	10.000000	10.000000	10.000000
mean	27.710000	0.507800	35.900000	0.600000
std	11.259016	0.654114	11.873874	0.516398
min	0.000000	0.134000	21.000000	0.000000
25%	25.850000	0.175500	29.250000	0.000000
50%	29.300000	0.240000	31.500000	1.000000
75%	32.950000	0.558000	45.750000	1.000000
max	43.100000	2.288000	54.000000	1.000000

Missing Values in Each Column:

Pregnancies 0

Glucose 0

BloodPressure 0

SkinThickness 0

Insulin 0

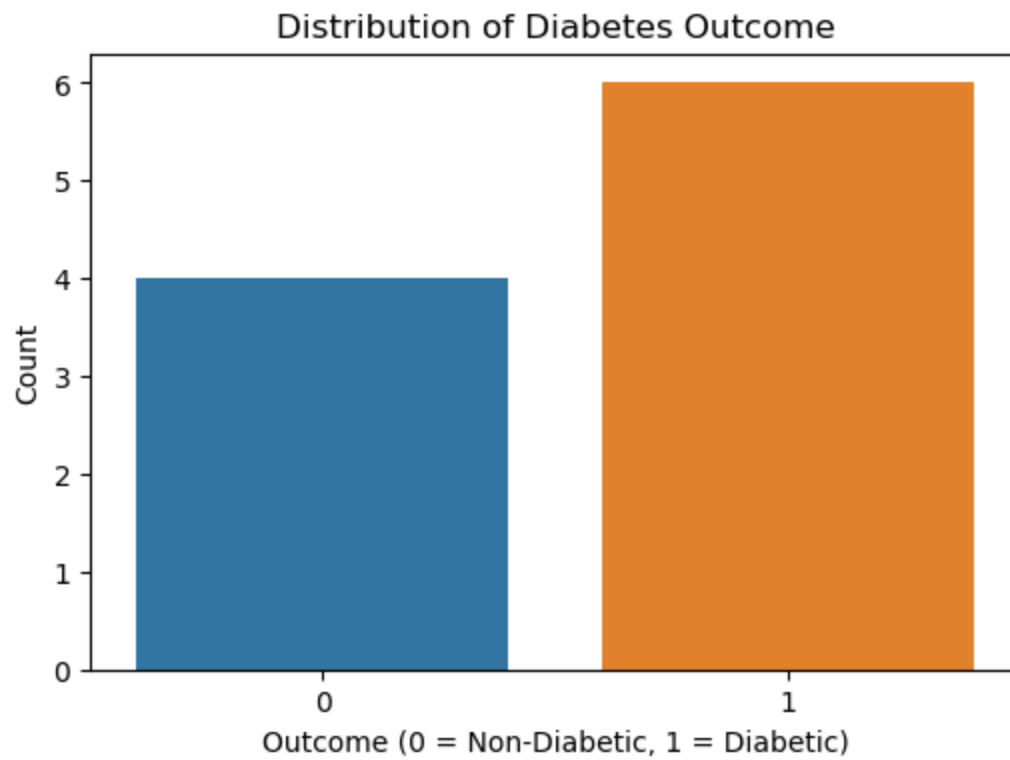
BMI 0

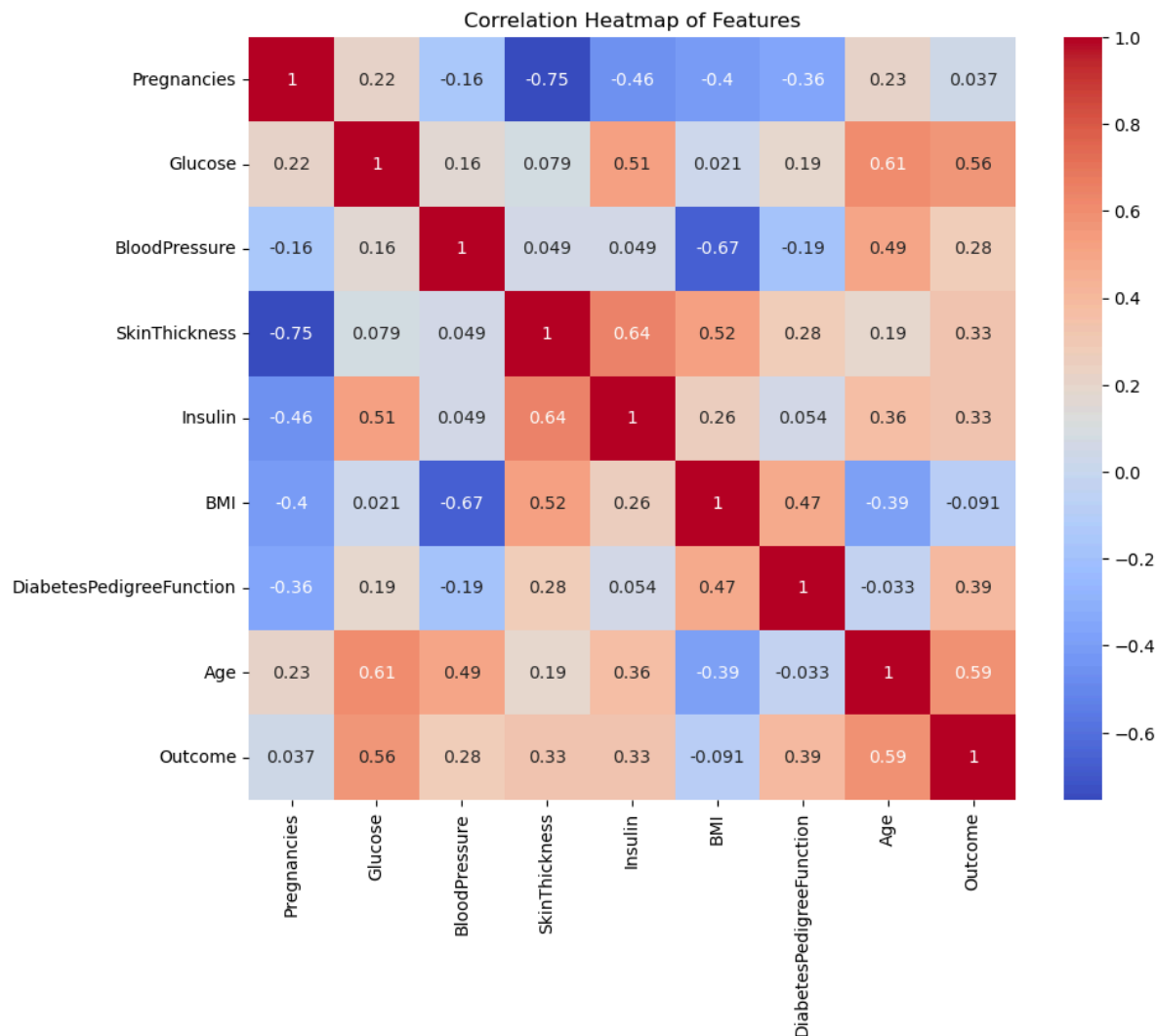
DiabetesPedigreeFunction 0

Age 0

Outcome 0

dtype: int64





2. Write a Python program to perform Time Series Analysis on a website traffic dataset. The program should load and clean the dataset, decompose the time series to identify trends and seasonality, visualize key metrics using moving averages and seasonal plots, and detect anomalies using statistical methods.

Dataset:

A dataset containing daily website traffic data with the following columns:

- Date – Date of observation
- Page_Views – Total number of page views per day
- Unique_Visitors – Number of unique visitors per day
- Bounce_Rate – Percentage of visitors who leave after viewing only one page (You can create a sample CSV file like website_traffic.csv for practice.)

Methodology:

1. Load and Clean Data:

- o Import the dataset and parse the Date column as a datetime object.
- o Handle missing values by interpolation.

2. Time Series Decomposition:

- o Decompose the time series data of Page_Views to identify trend, seasonality, and residuals.

3. Visualization:

- o Plot the time series data for Page_Views, Unique_Visitors, and

Bounce_Rate.

- o Use moving averages to smooth the time series and highlight trends.
- o Create seasonal plots to visualize patterns over time.

4. Anomaly Detection:

- o Identify and visualize anomalies in the time series data using statistical methods such as the Z-score.


```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
from scipy import stats

# --- Step 0: Generate Synthetic Website Traffic Data and Save as CSV ---

# Create a date range for 90 days
dates = pd.date_range(start='2023-01-01', periods=90, freq='D')

# Generate synthetic data with float arrays to allow NaN
np.random.seed(42)
page_views = (np.random.poisson(lam=500, size=90) + np.linspace(0, 100, 90)).astype(float)
unique_visitors = (page_views * np.random.uniform(0.6, 0.9, size=90)).astype(float)
bounce_rate = np.random.normal(loc=50, scale=10, size=90)

# Introduce some missing values
page_views[5] = np.nan
unique_visitors[20] = np.nan
bounce_rate[50] = np.nan

# Create dataframe
df_sample = pd.DataFrame({
    'Date': dates,
    'Page_Views': page_views,
    'Unique_Visitors': unique_visitors,
    'Bounce_Rate': bounce_rate
})

# Save to CSV
df_sample.to_csv('website_traffic.csv', index=False)
print("Sample 'website_traffic.csv' created successfully!")

# --- Step 1: Load and Clean Data ---

# Load dataset with Date parsing
df = pd.read_csv('website_traffic.csv', parse_dates=['Date'])

# Sort by Date
df = df.sort_values('Date')

# Set Date as index
df.set_index('Date', inplace=True)

# Handle missing values with interpolation
df = df.interpolate()

print("\nFirst few rows of the dataset:")
print(df.head())

print("\nDataset Information:")
print(df.info())

# --- Step 2: Time Series Decomposition (Page_Views) ---
```

```

decomposition = seasonal_decompose(df['Page_Views'], model='additive', period=7)

plt.figure(figsize=(12, 8))
decomposition.plot()
plt.suptitle('Time Series Decomposition of Page Views', fontsize=14)
plt.show()

# --- Step 3: Visualization ---

# (a) Plot all metrics
plt.figure(figsize=(14, 6))
plt.plot(df.index, df['Page_Views'], label='Page Views')
plt.plot(df.index, df['Unique_Visitors'], label='Unique Visitors')
plt.plot(df.index, df['Bounce_Rate'], label='Bounce Rate')
plt.title('Website Traffic Metrics Over Time')
plt.xlabel('Date')
plt.ylabel('Values')
plt.legend()
plt.show()

# (b) Moving Average on Page Views
df['PageViews_MA7'] = df['Page_Views'].rolling(window=7).mean()

plt.figure(figsize=(14, 5))
plt.plot(df.index, df['Page_Views'], label='Original')
plt.plot(df.index, df['PageViews_MA7'], label='7-Day Moving Average', linewidth=2)
plt.title('Page Views with Moving Average')
plt.legend()
plt.show()

# (c) Seasonal plot - Weekly pattern of Page Views
df['DayOfWeek'] = df.index.day_name()

plt.figure(figsize=(10, 5))
sns.boxplot(x='DayOfWeek', y='Page_Views', data=df.reset_index(),
            order=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])
plt.title('Seasonal Pattern by Day of Week')
plt.xticks(rotation=45)
plt.show()

# --- Step 4: Anomaly Detection using Z-score ---

df['Z_Score'] = np.abs(stats.zscore(df['Page_Views']))

# Define anomalies as points with Z-score > 3
anomalies = df[df['Z_Score'] > 3]

plt.figure(figsize=(14, 5))
plt.plot(df.index, df['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.title('Anomaly Detection in Page Views')
plt.legend()
plt.show()

print("\nDetected Anomalies:")
print(anomalies[['Page_Views', 'Z_Score']])

```

Sample 'website_traffic.csv' created successfully!

First few rows of the dataset:

	Page_Views	Unique_Visitors	Bounce_Rate
Date			
2023-01-01	492.000000	295.947090	42.696334
2023-01-02	517.123596	335.221449	52.164586
2023-01-03	476.247191	364.148192	50.455718
2023-01-04	509.370787	411.351832	43.483997
2023-01-05	528.494382	420.463988	71.439441

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

DatetimeIndex: 90 entries, 2023-01-01 to 2023-03-31

Data columns (total 3 columns):

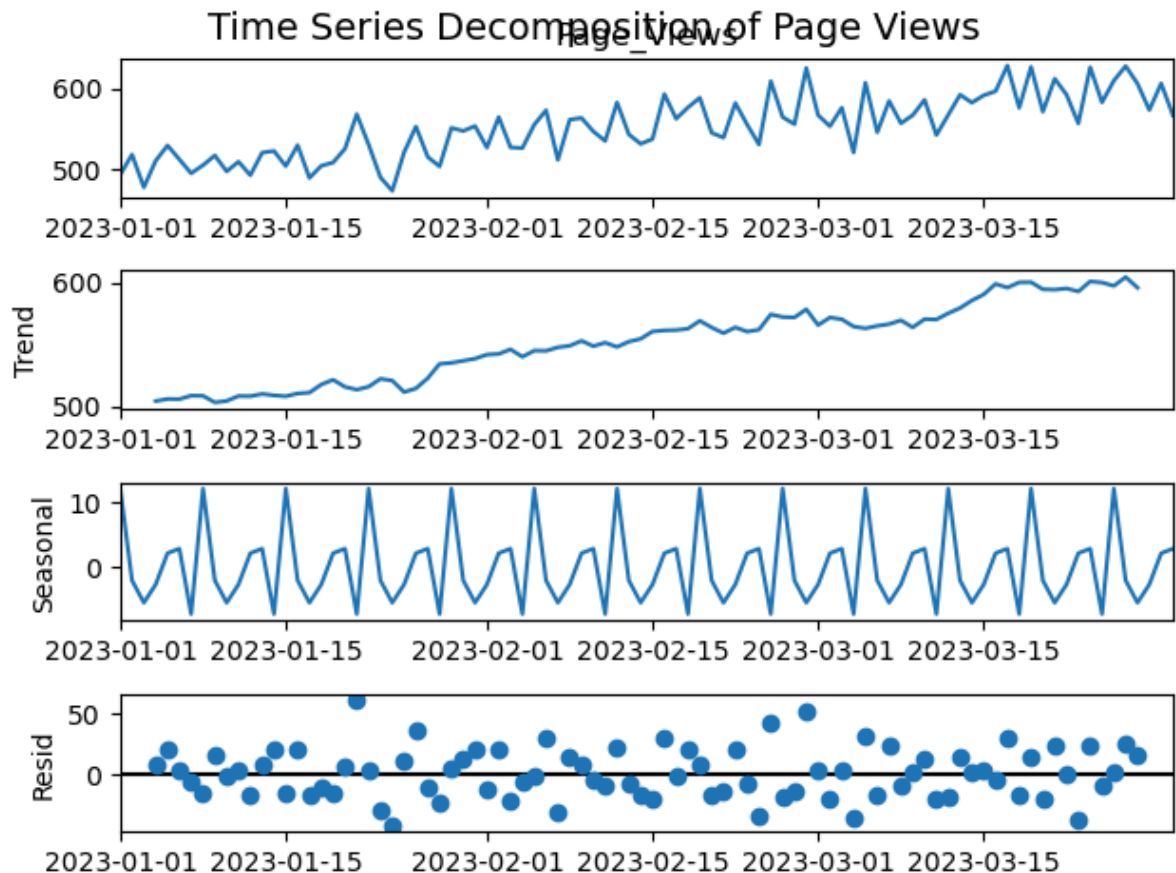
#	Column	Non-Null Count	Dtype
0	Page_Views	90 non-null	float64
1	Unique_Visitors	90 non-null	float64
2	Bounce_Rate	90 non-null	float64

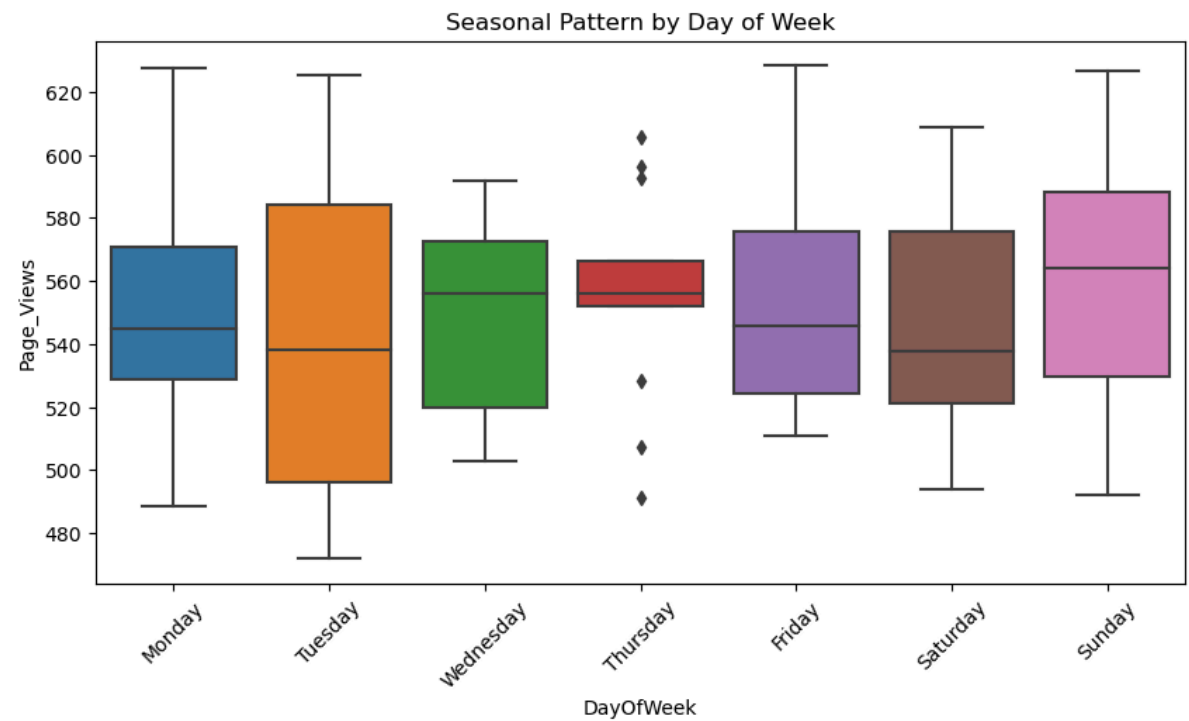
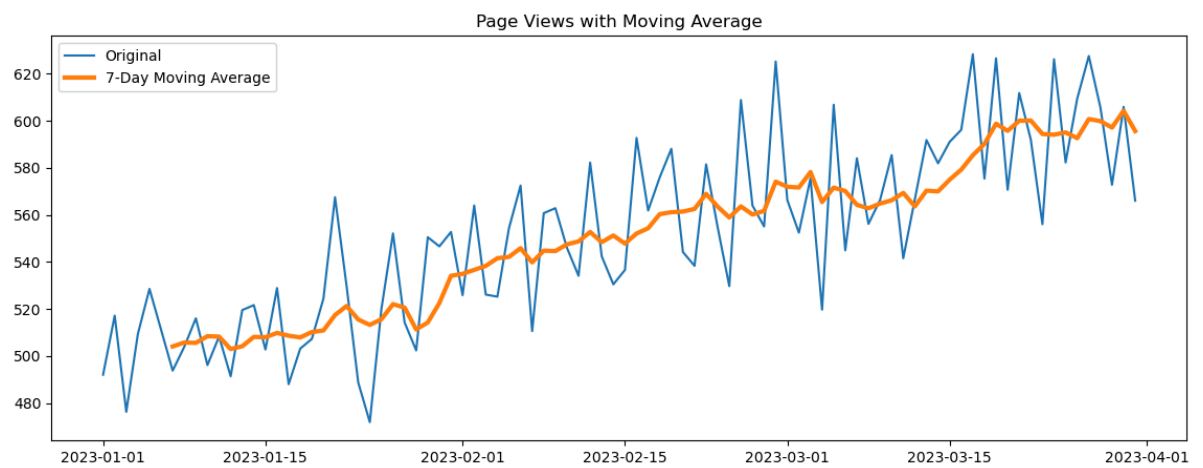
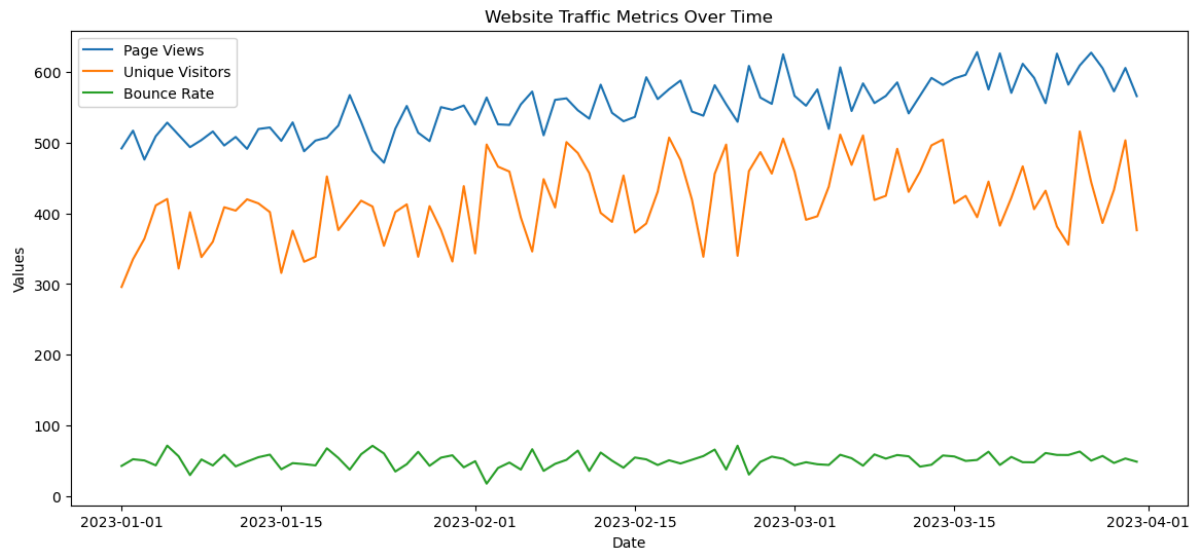
dtypes: float64(3)

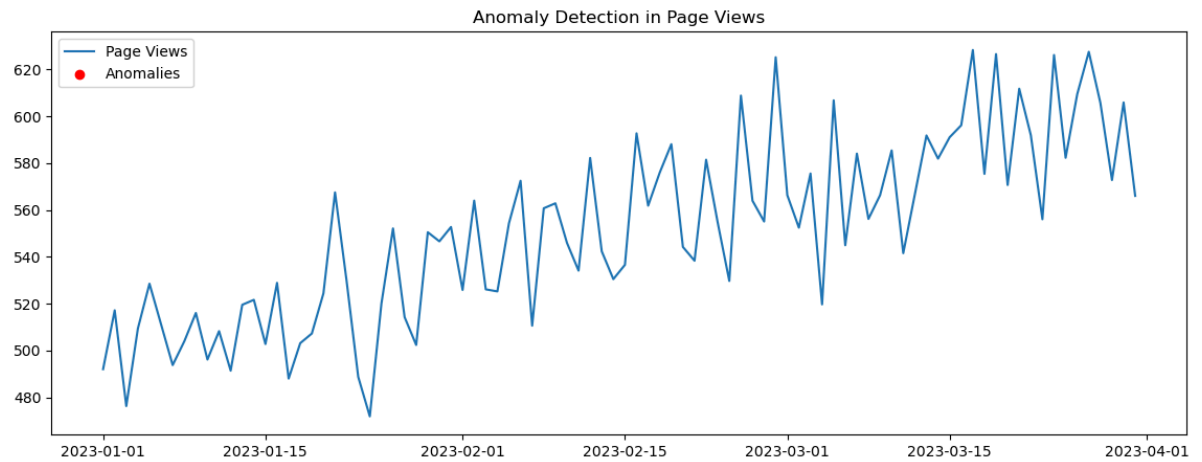
memory usage: 2.8 KB

None

<Figure size 1200x800 with 0 Axes>







Detected Anomalies:

Empty DataFrame

Columns: [Page_Views, Z_Score]

Index: []

3) Random Sampling and Sampling Distribution

To explore random sampling from a population and understand the concept of sampling

distribution using Python in Jupyter Notebook.

Steps:

1. Generate a Population:

- o Create a population of data with a specified distribution (e.g., normal distribution).

2. Random Sampling:

- o Perform random sampling from the population to create multiple samples of different sizes.
- o Compute sample statistics (mean, standard deviation, etc.) for each sample.

3. Sampling Distribution:

- o Plot histograms or density plots of sample statistics (e.g., sample means).
- o Compare the sampling distribution of the sample statistic (mean) with the population distribution.

4. Central Limit Theorem (Optional):

- o Demonstrate the Central Limit Theorem by showing that as sample size increases, the sampling distribution of the sample mean approaches a normal distribution regardless of the population distribution.


```

In [8]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Generate a Population
np.random.seed(42) # For reproducibility

population_size = 100000
# Population from a normal distribution with mean=50, std=15
population = np.random.normal(loc=50, scale=15, size=population_size)

plt.figure(figsize=(10, 5))
sns.histplot(population, bins=50, kde=True, color='skyblue')
plt.title("Population Distribution (Normal Distribution)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

# Step 2 & 3: Random Sampling and Sampling Distribution
sample_sizes = [5, 30, 100]
num_samples = 1000 # Number of samples to draw for each sample size

plt.figure(figsize=(15, 12))

for i, size in enumerate(sample_sizes, 1):
    sample_means = []

    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means.append(np.mean(sample))

    plt.subplot(3, 1, i)
    sns.histplot(sample_means, bins=30, kde=True, color='coral')
    plt.title(f"Sampling Distribution of the Sample Mean (Sample Size = {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.axvline(np.mean(population), color='blue', linestyle='--', label='Population Mean')
    plt.legend()

plt.tight_layout()
plt.show()

# Step 4: Central Limit Theorem Demonstration with Non-Normal Population
# Generate a population with a uniform distribution (non-normal)
uniform_population = np.random.uniform(low=0, high=100, size=population_size)

plt.figure(figsize=(10, 5))
sns.histplot(uniform_population, bins=50, kde=True, color='lightgreen')
plt.title("Population Distribution (Uniform Distribution)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

plt.figure(figsize=(15, 12))

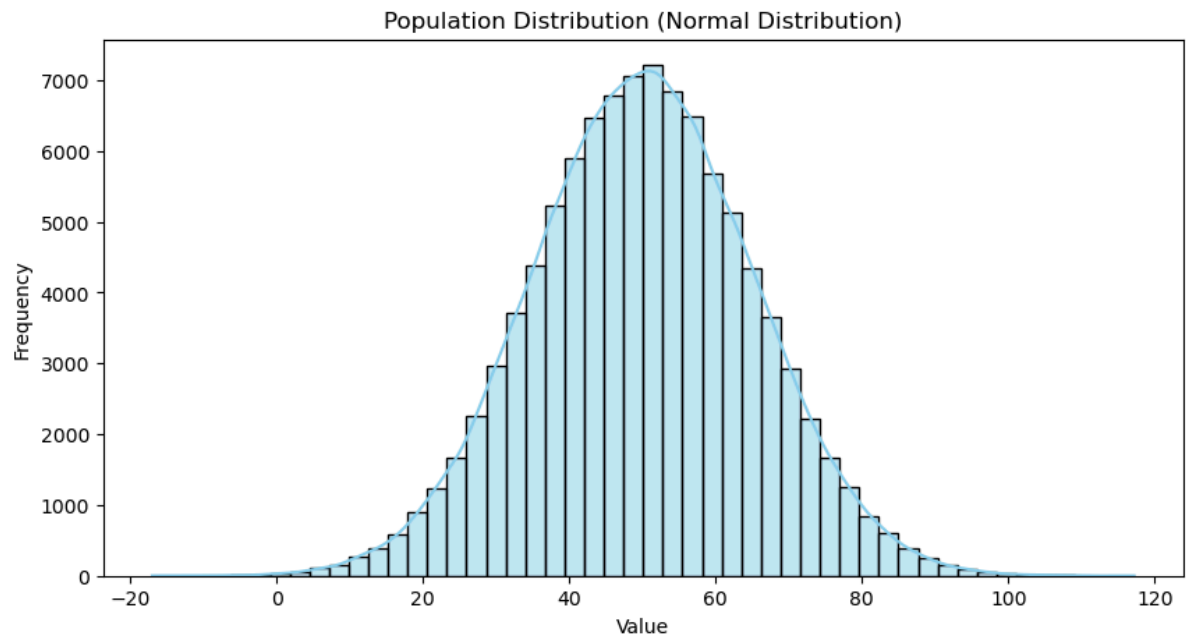
for i, size in enumerate(sample_sizes, 1):
    sample_means = []

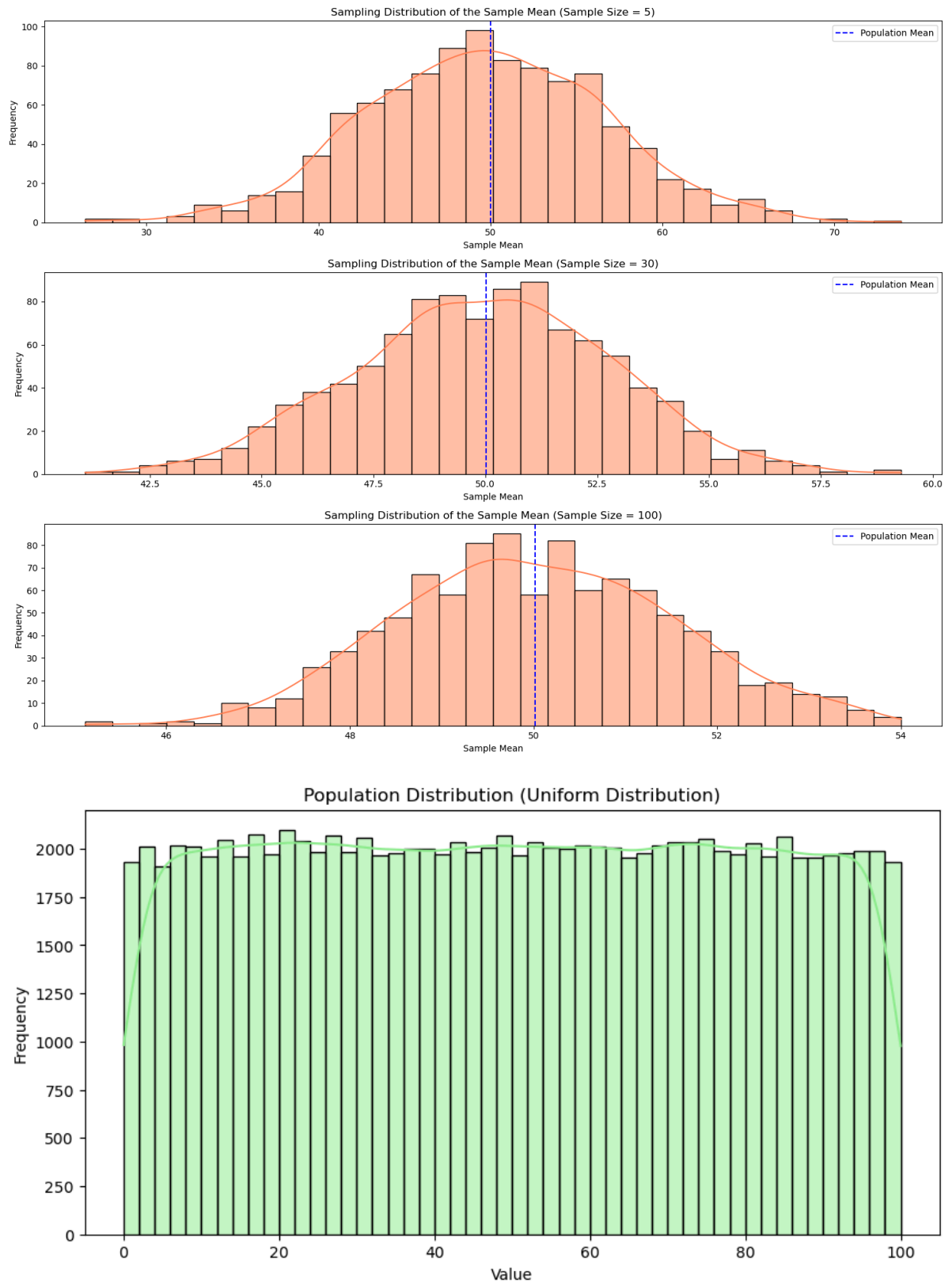
```

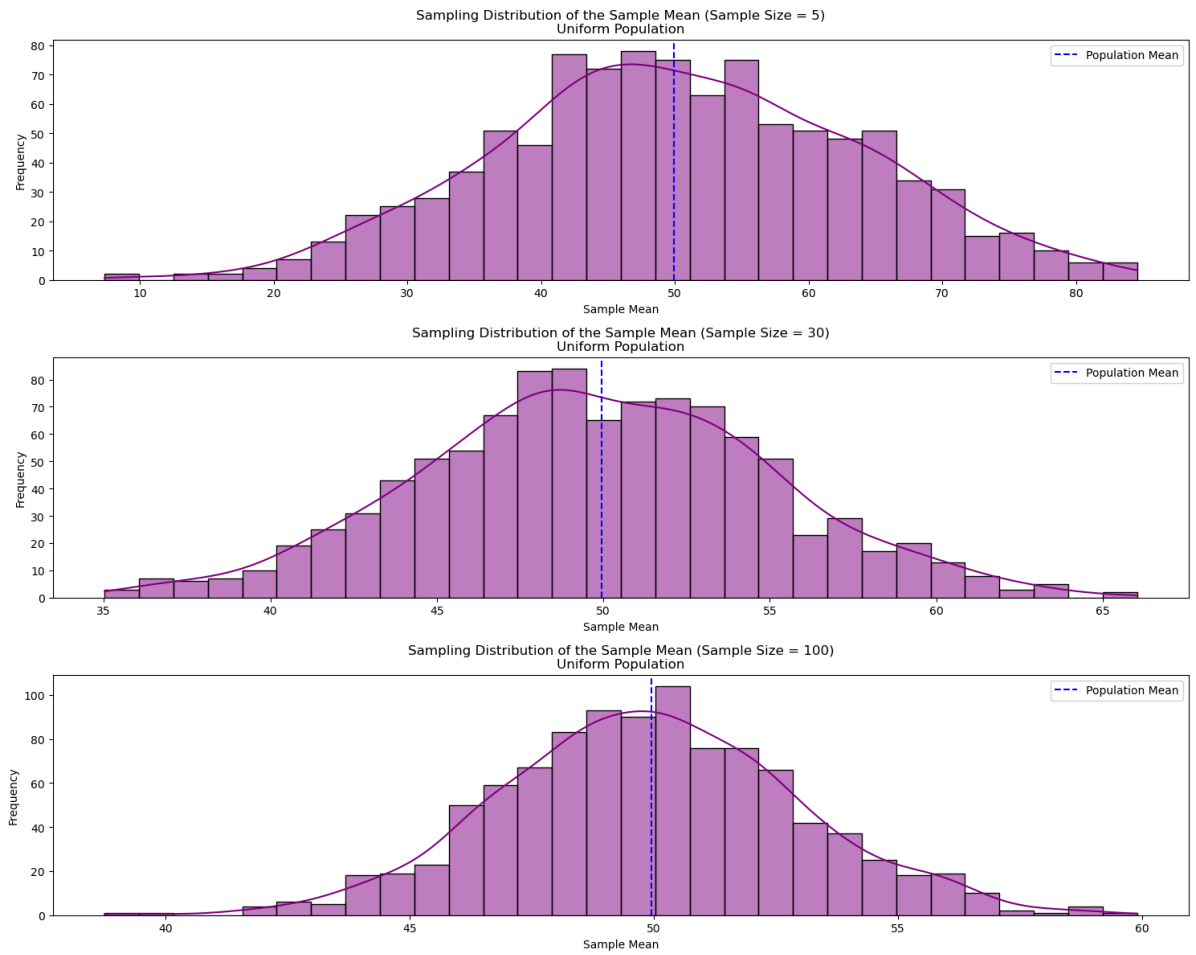
```
for _ in range(num_samples):
    sample = np.random.choice(uniform_population, size=size, replace=False)
    sample_means.append(np.mean(sample))

plt.subplot(3, 1, i)
sns.histplot(sample_means, bins=30, kde=True, color='purple')
plt.title(f"Sampling Distribution of the Sample Mean (Sample Size = {size})")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.axvline(np.mean(uniform_population), color='blue', linestyle='--', label=)
plt.legend()

plt.tight_layout()
plt.show()
```







Lab Experiment 8: Conduct Z test for the Data Given using Python Objective: To test whether the average weight of a species of birds differs from 150 grams.

```
import numpy as np
import scipy.stats as stats

sample_data =
np.array([152,148,151,149,147,153,150,148,152,149,151,150,149,152,151,
148,150,152,149,150,148,153,151,150,149,152,148,151,150,153])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std /
np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is
significantly different from 150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant
difference in average weight from 150 grams.")

Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference
in average weight from 150 grams.
```

Exp No:9 Experiment to understand Matplotlib library use cases in Data Science through visualization

Description: Use Iris data set to understand the Matplotlib library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA

iris = datasets.load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
```

```

df['species'] = pd.Categorical.from_codes(iris.target,
iris.target_names)

plt.figure(figsize=(6,4))
scatter = plt.scatter(df['sepal length (cm)'], df['sepal width (cm)'],
c=iris.target, cmap='viridis', edgecolor='k')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Sepal Length vs Width – Iris Dataset')
handles, _ = scatter.legend_elements()
plt.legend(handles, iris.target_names, title="Species", loc="best")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.hist(df['petal length (cm)'], bins=15, color='skyblue',
edgecolor='black')
plt.xlabel('Petal Length (cm)')
plt.title('Distribution of Petal Length – Iris Dataset')
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
positions = list(range(len(df['species'].cat.categories)))
data = [group['petal width (cm)'].values for species, group in
df.groupby('species')]
plt.boxplot(data, positions=positions, widths=0.6)
plt.xticks(positions, df['species'].cat.categories)
plt.xlabel('Species')
plt.ylabel('Petal Width (cm)')
plt.title('Petal Width by Species – Iris Dataset')
plt.tight_layout()
plt.show()

X = iris.data
y = iris.target
X_reduced = PCA(n_components=3).fit_transform(X)
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d', elev=-150, azimuth=110)
ax.scatter(X_reduced[:,0], X_reduced[:,1], X_reduced[:,2], c=y,
cmap='viridis', edgecolor='k', s=50)
ax.set_title('First three PCA components of Iris Dataset')
ax.set_xlabel('PCA1')
ax.set_ylabel('PCA2')
ax.set_zlabel('PCA3')
plt.tight_layout()
plt.show()

sns.set_theme(style="whitegrid")
sns.pairplot(df, hue='species', palette='viridis', diag_kind='kde',

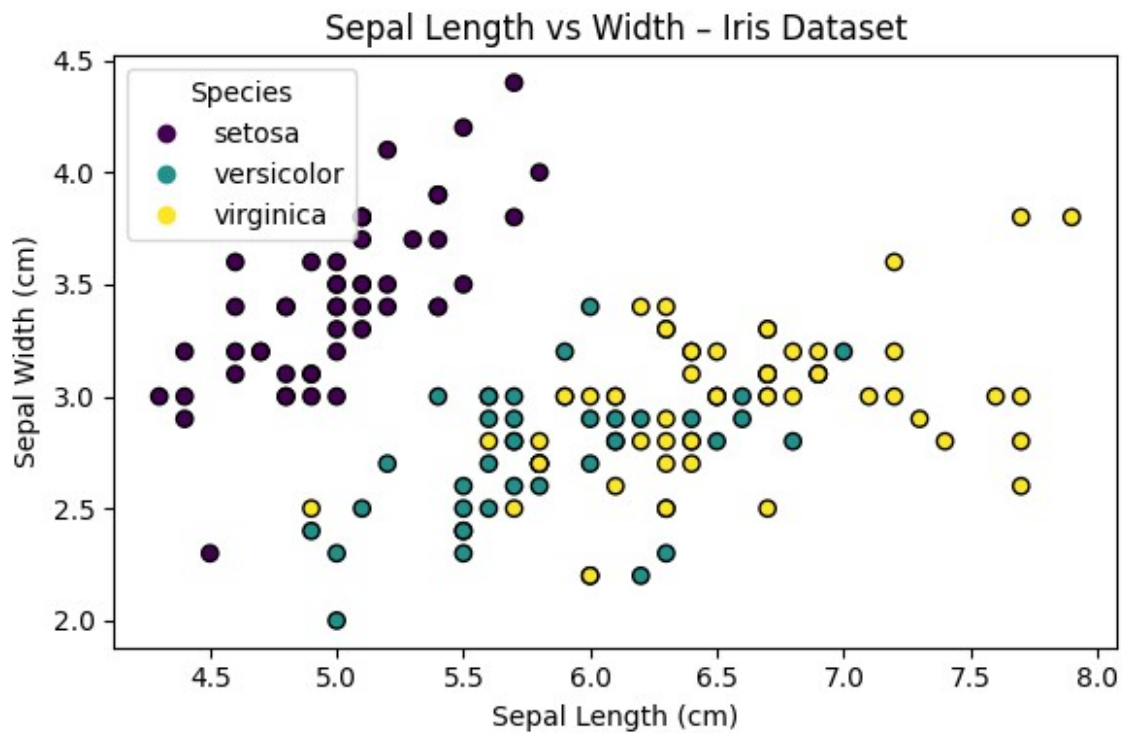
```

```

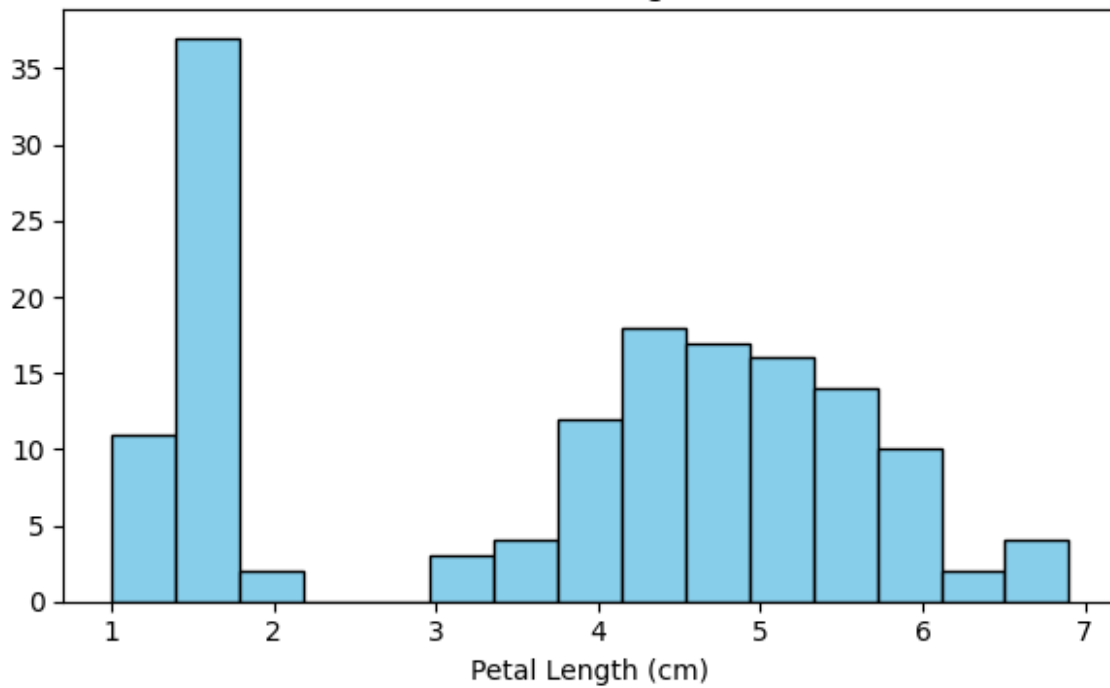
height=2.5)
plt.show()

g = sns.PairGrid(df, hue='species', palette='viridis',
diag_sharey=False, height=2.5)
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot, fill=True)
g.map_diag(sns.histplot, kde=True)
g.add_legend()
plt.show()

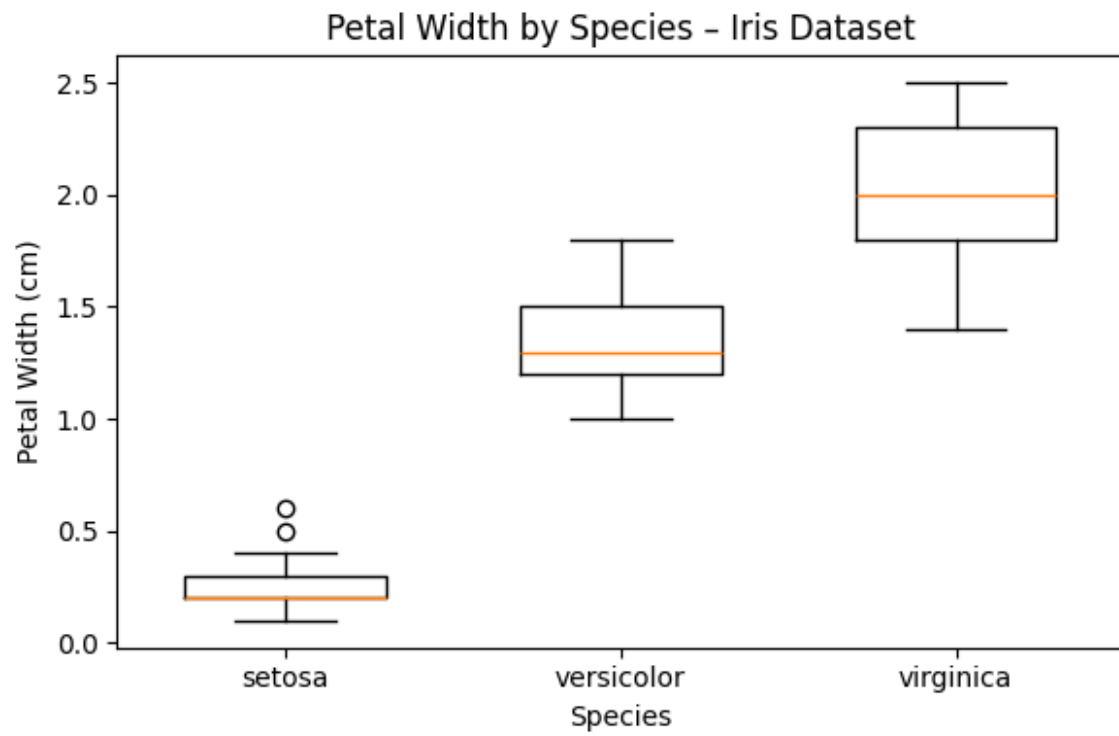
```



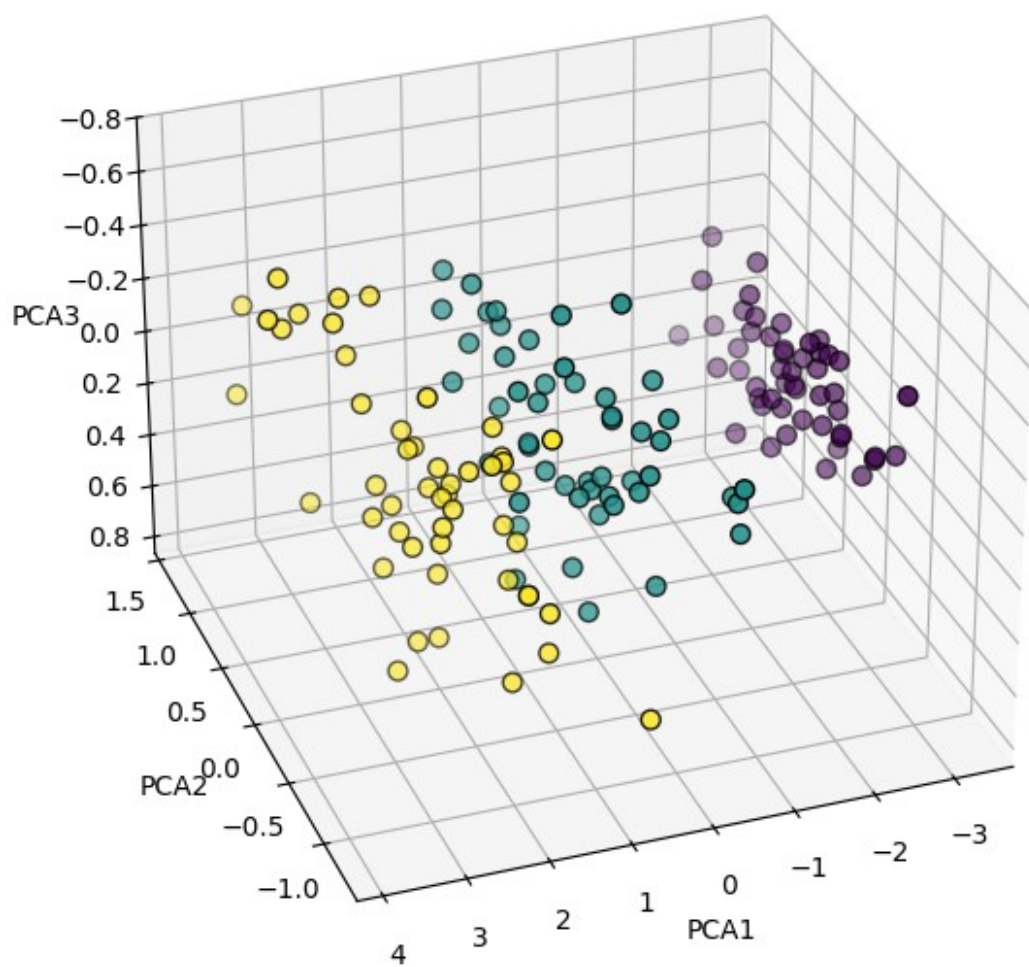
Distribution of Petal Length - Iris Dataset

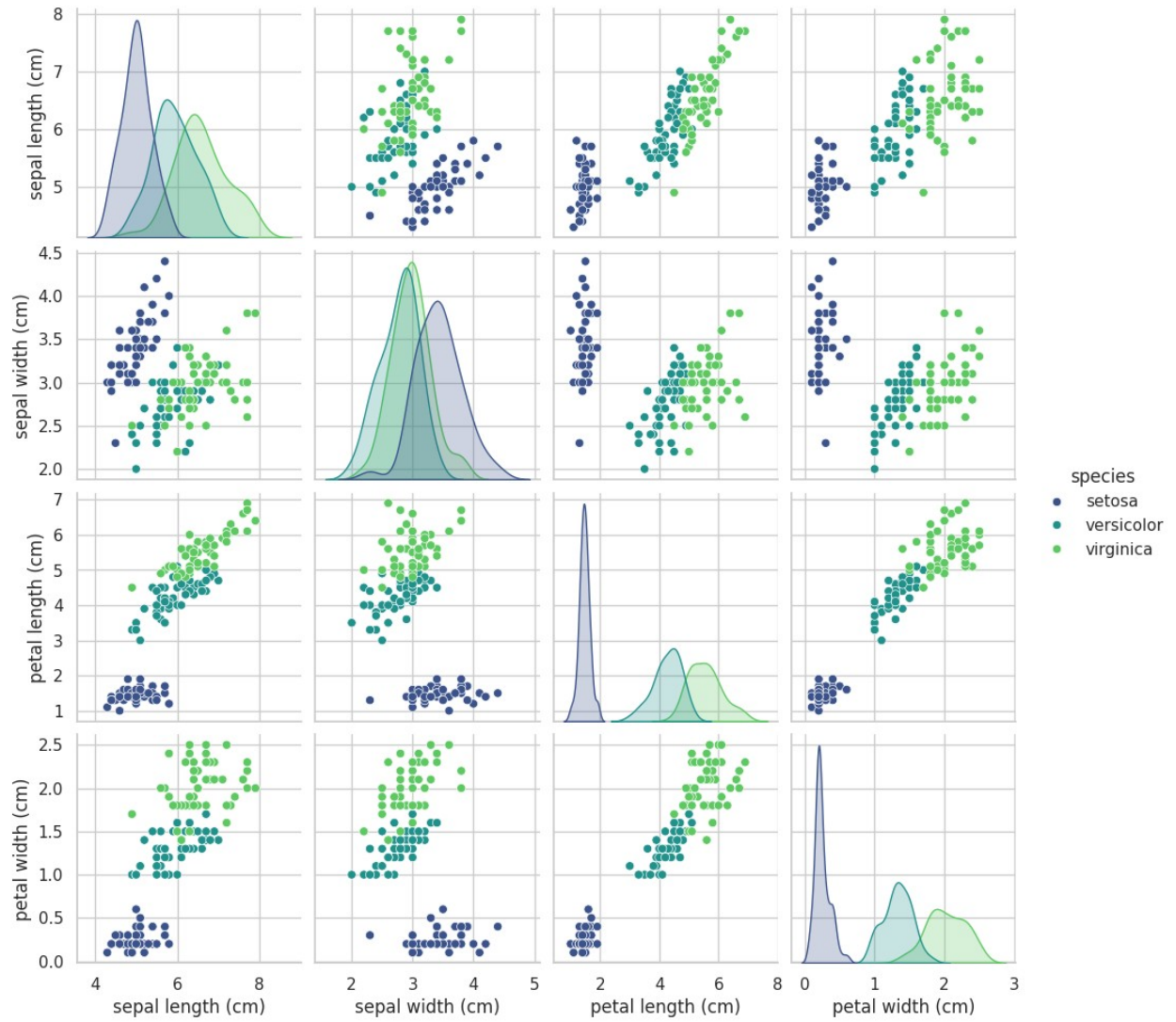


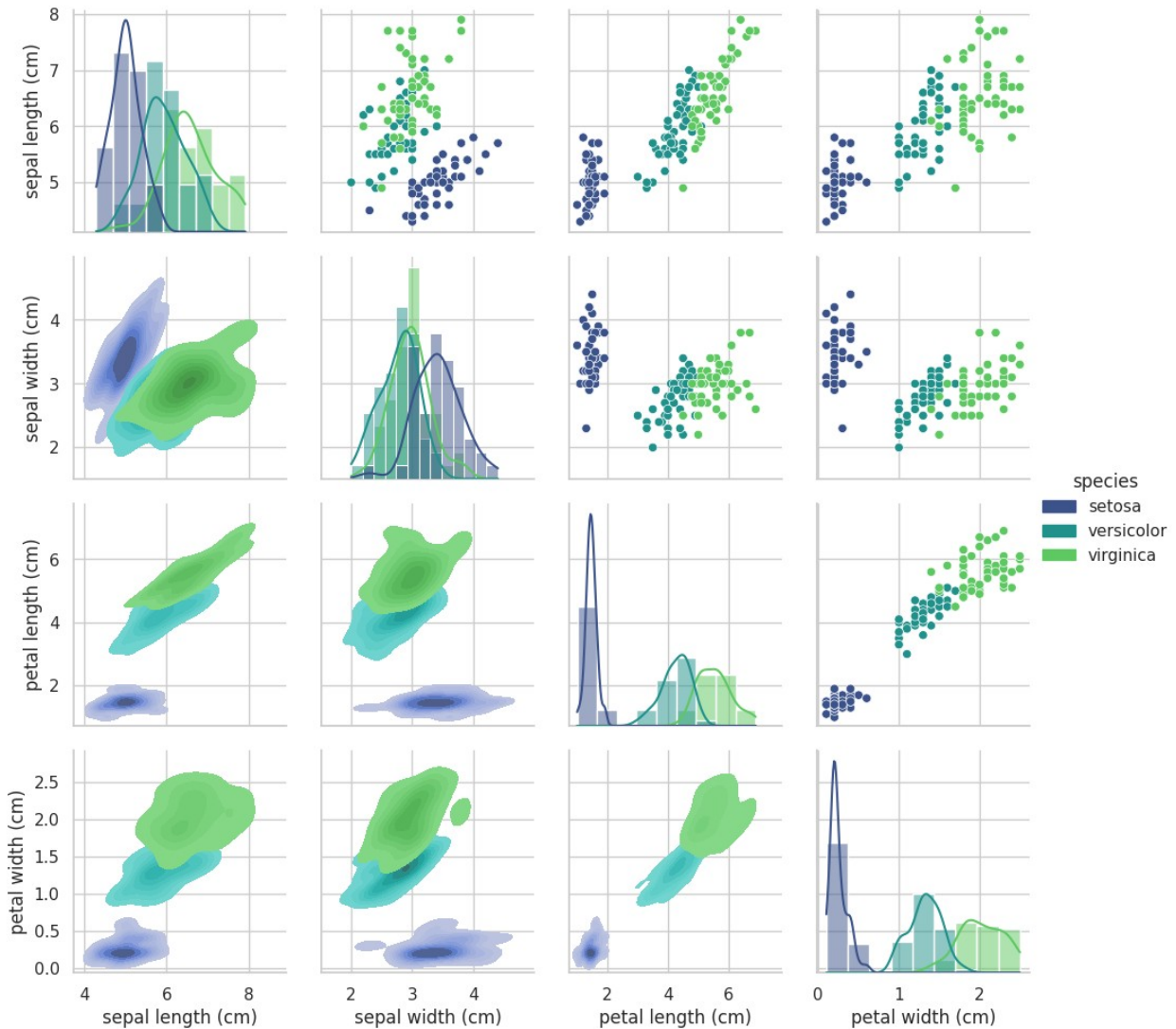
```
/tmp/ipython-input-625443151.py:35: FutureWarning: The default of
observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or
observed=True to adopt the future default and silence this warning.
  data = [group['petal width (cm)'].values for species, group in
df.groupby('species')]
```



First three PCA components of Iris Dataset







Exp No:10 Experiment to understand array function in Data science. Description: Understand array function using Numpy library

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.arange(10, 20, 2)
c = np.linspace(0, 1, 5)
d = np.zeros((2,3))
e = np.ones((3,2))
f = np.random.rand(2,2)

g = b.reshape((5,1))
h = d.flatten()
i = e.transpose()
j = np.concatenate((a, [6,7,8]))
k = np.vstack((d, np.ones((1,3))))
```

```

l = np.hstack((e, np.zeros((3,1))))

sum_a = np.sum(a)
mean_b = np.mean(b)
max_c = np.max(c)
min_c = np.min(c)
sqrt_a = np.sqrt(a)
exp_a = np.exp(a)
clip_b = np.clip(b, 12, 16)

first_elem = a[0]
slice_a = a[1:4]
col_from_matrix = k[:,1]
bool_idx = b[b > 14]
where_idx = np.where(b > 14)

sorted_b = np.sort(b)
argmax_b = np.argmax(b)
unique_vals = np.unique(np.array([1,2,2,3,3,3]))

m = a * 2
n = a + np.array([10,10,10,10,10])
o = a * b[:5]

print("a:", a)
print("b:", b)
print("g (reshaped b):", g)
print("sum_a:", sum_a, "mean_b:", mean_b)
print("bool_idx (b>14):", bool_idx)
print("sorted_b:", sorted_b, "argmax_b:", argmax_b)
print("unique_vals:", unique_vals)
print("o (a * b[:5]):", o)

a: [1 2 3 4 5]
b: [10 12 14 16 18]
g (reshaped b): [[10]
 [12]
 [14]
 [16]
 [18]]
sum_a: 15 mean_b: 14.0
bool_idx (b>14): [16 18]
sorted_b: [10 12 14 16 18] argmax_b: 4
unique_vals: [1 2 3]
o (a * b[:5]): [10 24 42 64 90]

```

Exp No:11 Experiment to understand pandas library use cases in Data science. Description: Understand data frame use cases using pandas library

```

import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Evan', 'Fiona'],
    'Age': [25, 30, 35, 28, 40, np.nan],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los
Angeles', 'Chicago'],
    'Score': [85, 92, 78, 88, np.nan, 95]
}

df = pd.DataFrame(data)
df['Passed'] = df['Score'] >= 80

df_clean = df.dropna(subset=['Age', 'Score'])
df_filled = df.fillna({'Age': df['Age'].mean(), 'Score':
df['Score'].mean()})

city_group = df_filled.groupby('City').agg({
    'Age': ['mean', 'min', 'max'],
    'Score': ['mean', 'count']
})

df_sorted = df_filled.sort_values(by='Score', ascending=False)
df_selected = df_filled.loc[df_filled['Passed'] == True,
['Name', 'Score', 'City']]

df_merged = pd.concat([df_selected.reset_index(drop=True),
                        pd.DataFrame({'Bonus': [5,5,5,5,5,5]})],
axis=1)

df_merged.to_csv('output_scores.csv', index=False)

print("Original DataFrame:")
print(df)
print("\nCleaned DataFrame:")
print(df_clean)
print("\nFilled DataFrame:")
print(df_filled)
print("\nGroup by City:")
print(city_group)
print("\nSorted by Score:")
print(df_sorted)
print("\nSelected Passed Students:")
print(df_selected)
print("\nMerged with Bonus:")
print(df_merged)

```

Original DataFrame:

Name	Age	City	Score	Passed
------	-----	------	-------	--------

0	Alice	25.0	New York	85.0	True
1	Bob	30.0	Los Angeles	92.0	True
2	Charlie	35.0	Chicago	78.0	False
3	Diana	28.0	New York	88.0	True
4	Evan	40.0	Los Angeles	NaN	False
5	Fiona	NaN	Chicago	95.0	True

Cleaned DataFrame:

	Name	Age	City	Score	Passed
0	Alice	25.0	New York	85.0	True
1	Bob	30.0	Los Angeles	92.0	True
2	Charlie	35.0	Chicago	78.0	False
3	Diana	28.0	New York	88.0	True

Filled DataFrame:

	Name	Age	City	Score	Passed
0	Alice	25.0	New York	85.0	True
1	Bob	30.0	Los Angeles	92.0	True
2	Charlie	35.0	Chicago	78.0	False
3	Diana	28.0	New York	88.0	True
4	Evan	40.0	Los Angeles	87.6	False
5	Fiona	31.6	Chicago	95.0	True

Group by City:

City	Age			Score	
	mean	min	max	mean	count
Chicago	33.3	31.6	35.0	86.5	2
Los Angeles	35.0	30.0	40.0	89.8	2
New York	26.5	25.0	28.0	86.5	2

Sorted by Score:

	Name	Age	City	Score	Passed
5	Fiona	31.6	Chicago	95.0	True
1	Bob	30.0	Los Angeles	92.0	True
3	Diana	28.0	New York	88.0	True
4	Evan	40.0	Los Angeles	87.6	False
0	Alice	25.0	New York	85.0	True
2	Charlie	35.0	Chicago	78.0	False

Selected Passed Students:

	Name	Score	City
0	Alice	85.0	New York
1	Bob	92.0	Los Angeles
3	Diana	88.0	New York
5	Fiona	95.0	Chicago

Merged with Bonus:

	Name	Score	City	Bonus
0	Alice	85.0	New York	5

1	Bob	92.0	Los Angeles	5
2	Diana	88.0	New York	5
3	Fiona	95.0	Chicago	5
4	NaN	NaN	NaN	5
5	NaN	NaN	NaN	5

Exp No:12 Experiment to detect outliers in a given data set. Description: Understand the procedure to identify the outliers in a given dataset

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'feature1': [10, 12, 13, 12, 11, 14, 100, 12, 11, 13],
    'feature2': [1.2, 1.3, 1.1, 1.4, 1.3, 1.2, 1.0, 5.0, 1.3, 1.2]
}
df = pd.DataFrame(data)

mean_f1 = df['feature1'].mean()
std_f1 = df['feature1'].std(ddof=0)
df['z_score_f1'] = (df['feature1'] - mean_f1) / std_f1
outliers_z = df[np.abs(df['z_score_f1']) > 3]

Q1 = df['feature2'].quantile(0.25)
Q3 = df['feature2'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers_iqr = df[(df['feature2'] < lower_bound) | (df['feature2'] >
upper_bound)]

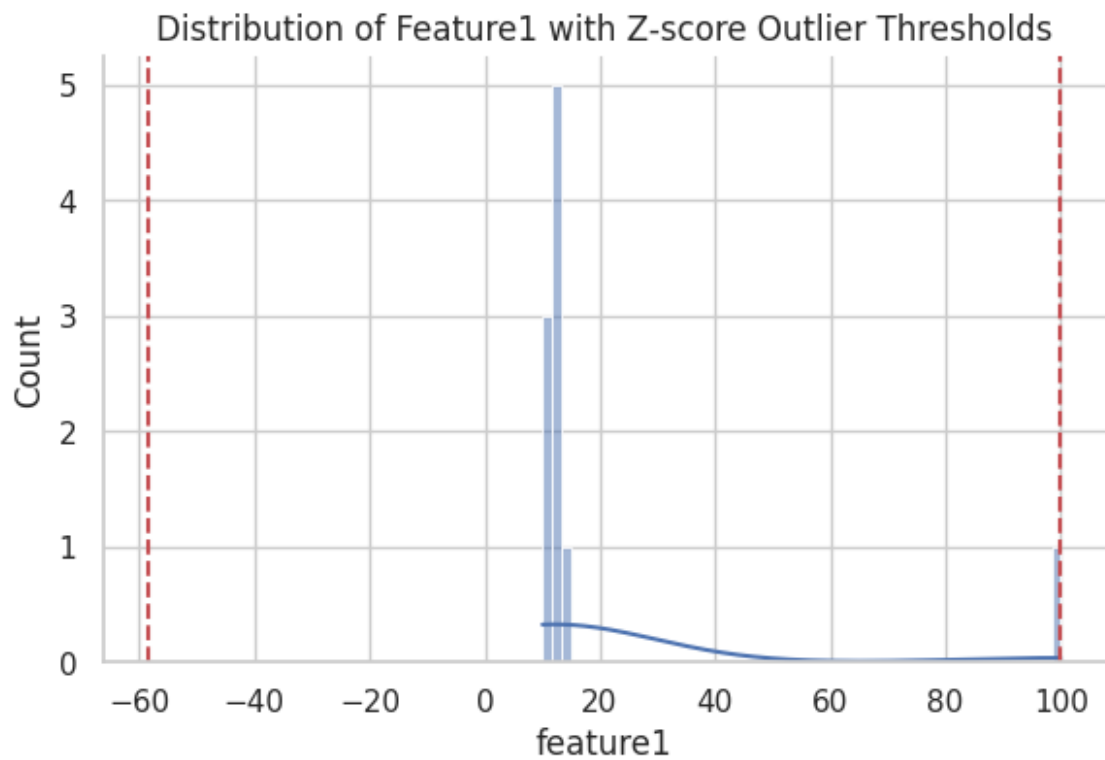
df['outlier_flag_z'] = np.where(np.abs(df['z_score_f1']) > 3, True,
False)
df['outlier_flag_iqr'] = np.where((df['feature2'] < lower_bound) |
(df['feature2'] > upper_bound), True, False)

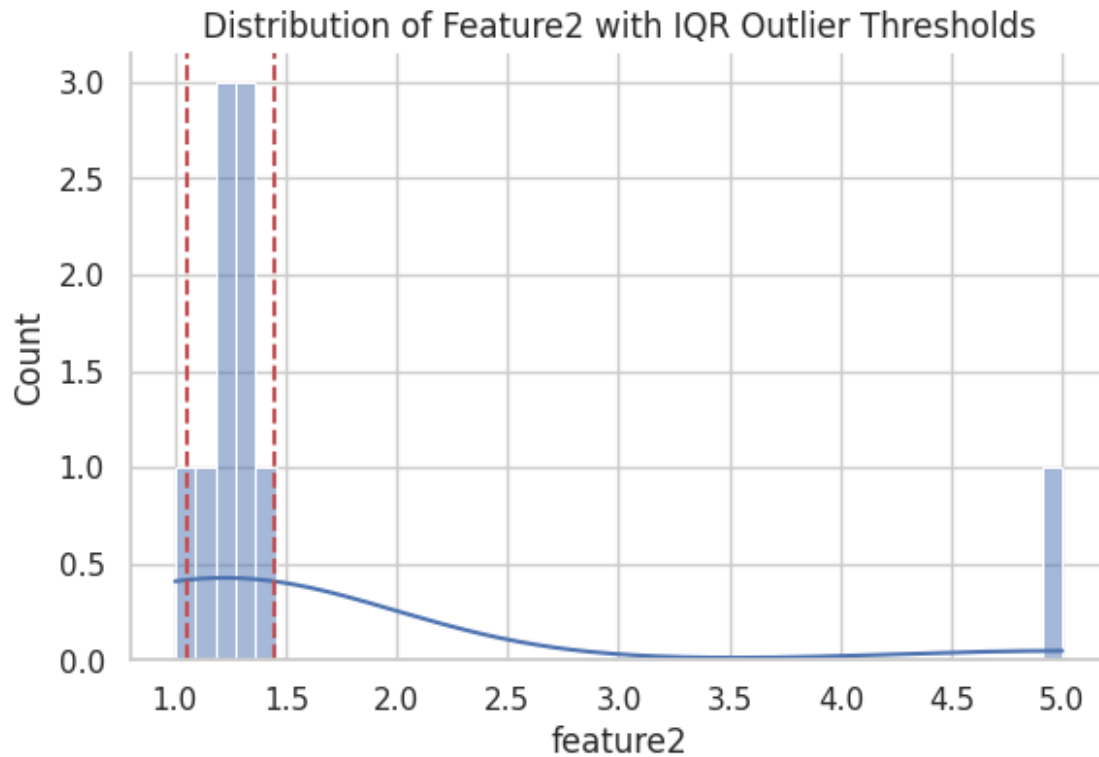
sns.set_style('whitegrid')
sns.displot(df['feature1'], kde=True, height=4, aspect=1.5)
plt.axvline(mean_f1 + 3 * std_f1, color='r', linestyle='--')
plt.axvline(mean_f1 - 3 * std_f1, color='r', linestyle='--')
plt.title('Distribution of Feature1 with Z-score Outlier Thresholds')
plt.show()

sns.displot(df['feature2'], kde=True, height=4, aspect=1.5)
plt.axvline(lower_bound, color='r', linestyle='--')
plt.axvline(upper_bound, color='r', linestyle='--')
plt.title('Distribution of Feature2 with IQR Outlier Thresholds')
```

```
plt.show()

print("DataFrame:")
print(df)
print("\nOutliers detected by Z-score method (feature1):")
print(outliers_z)
print("\nOutliers detected by IQR method (feature2):")
print(outliers_iqr)
```





DataFrame:

	feature1	feature2	z_score_f1	outlier_flag_z	outlier_flag_iqr
0	10	1.2	-0.408739	False	False
1	12	1.3	-0.333047	False	False
2	13	1.1	-0.295201	False	False
3	12	1.4	-0.333047	False	False
4	11	1.3	-0.370893	False	False
5	14	1.2	-0.257354	False	False
6	100	1.0	2.997421	False	True
7	12	5.0	-0.333047	False	True
8	11	1.3	-0.370893	False	False
9	13	1.2	-0.295201	False	False

Outliers detected by Z-score method (feature1):

Empty DataFrame

Columns: [feature1, feature2, z_score_f1]

Index: []

Outliers detected by IQR method (feature2):

	feature1	feature2	z_score_f1
6	100	1.0	2.997421
7	12	5.0	-0.333047

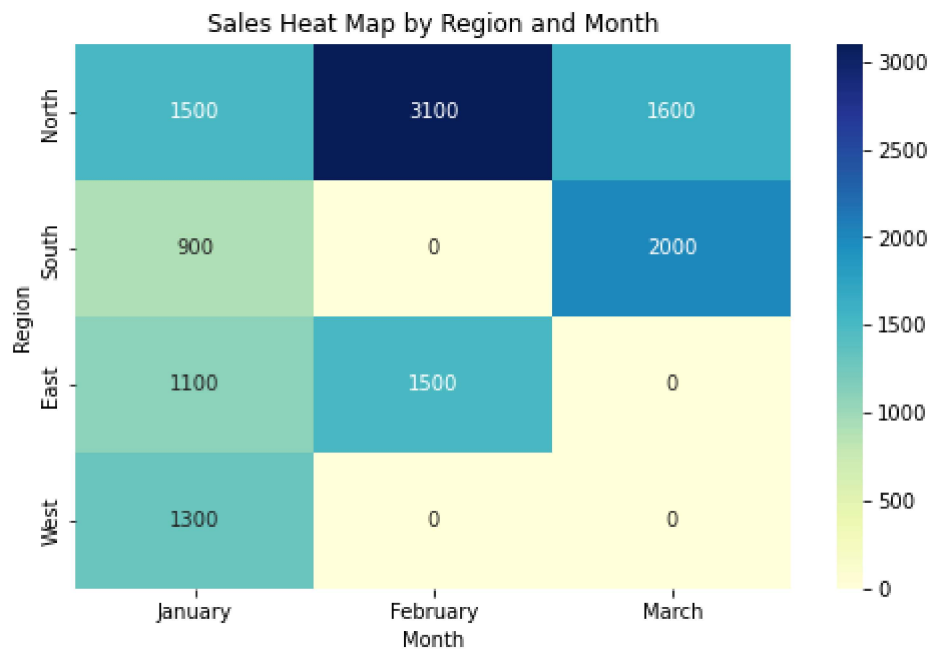
```

In [28]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = {
    'Region': ['North', 'South', 'East', 'West'],
    'January': [1500, 900, 1100, 1300],
    'February': [3100, 0, 1500, 0],
    'March': [1600, 2000, 0, 0]
}
df = pd.DataFrame(data)
df.set_index('Region', inplace=True)
print("Sales Data (in units):")
print(df)
print("\n")
plt.figure(figsize=(8, 5))
sns.heatmap(df, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Sales Heat Map by Region and Month')
plt.xlabel('Month')
plt.ylabel('Region')
plt.show()
region_totals = df.sum(axis=1)
month_totals = df.sum(axis=0)
print("Total Sales by Region:")
print(region_totals)
print("\nTotal Sales by Month:")
print(month_totals)
print("\n")
max_region = region_totals.idxmax()
min_region = region_totals.idxmin()
max_month = month_totals.idxmax()
min_month = month_totals.idxmin()
print(f"Region with highest total sales: {max_region} ({region_totals[max_region]})")
print(f"Region with lowest total sales: {min_region} ({region_totals[min_region]})")
print(f"Month with highest total sales: {max_month} ({month_totals[max_month]})")
print(f"Month with lowest total sales: {min_month} ({month_totals[min_month]})")

```

Sales Data (in units):

	January	February	March
Region			
North	1500	3100	1600
South	900	0	2000
East	1100	1500	0
West	1300	0	0



Total Sales by Region:

Region

North 6200

South 2900

East 2600

West 1300

dtype: int64

Total Sales by Month:

January 4800

February 4600

March 3600

dtype: int64

Region with highest total sales: North (6200)

Region with lowest total sales: West (1300)

Month with highest total sales: January (4800)

Month with lowest total sales: March (3600)

```

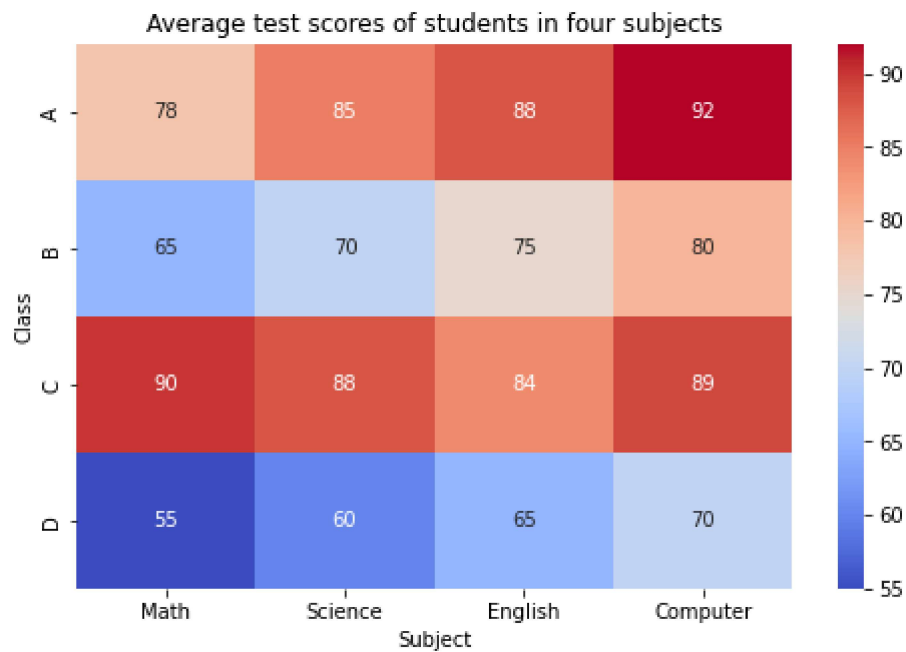
In [29]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = {
    'Class': ['A', 'B', 'C', 'D'],
    'Math': [78, 65, 90, 55],
    'Science': [85, 70, 88, 60],
    'English': [88, 75, 84, 65],
    'Computer': [92, 80, 89, 70]
}
df = pd.DataFrame(data)
df.set_index('Class', inplace=True)
print("Class :")
print(df)
print("\n")
plt.figure(figsize=(8, 5))
sns.heatmap(df, annot=True, cmap='coolwarm', fmt='.0f')
plt.title('Average test scores of students in four subjects')
plt.xlabel('Subject')
plt.ylabel('Class')
plt.show()
Class_totals = df.sum(axis=1)
Subject_totals = df.sum(axis=0)
print("Class with average score:")
print(Class_totals)
print("\nSubject with average score:")
print(Subject_totals)
print("\n")
max_Class = Class_totals.idxmax()
max_Subject = Subject_totals.idxmax()
print(f"Subject with highest overall average score: {max_Subject} ({Subject_totals[max_Subject]})")
print(f"Class with highest Score: {max_Class} ({Class_totals[max_Class]})")

```

```

Class :
      Math  Science  English  Computer
Class
A         78       85       88       92
B         65       70       75       80
C         90       88       84       89
D         55       60       65       70

```



Class with average score:

Class

A 343

B 290

C 351

D 250

dtype: int64

Subject with average score:

Math 288

Science 303

English 312

Computer 331

dtype: int64

Subject with highest overall average score: Computer (331)

Class with highest Score: C (351)

```

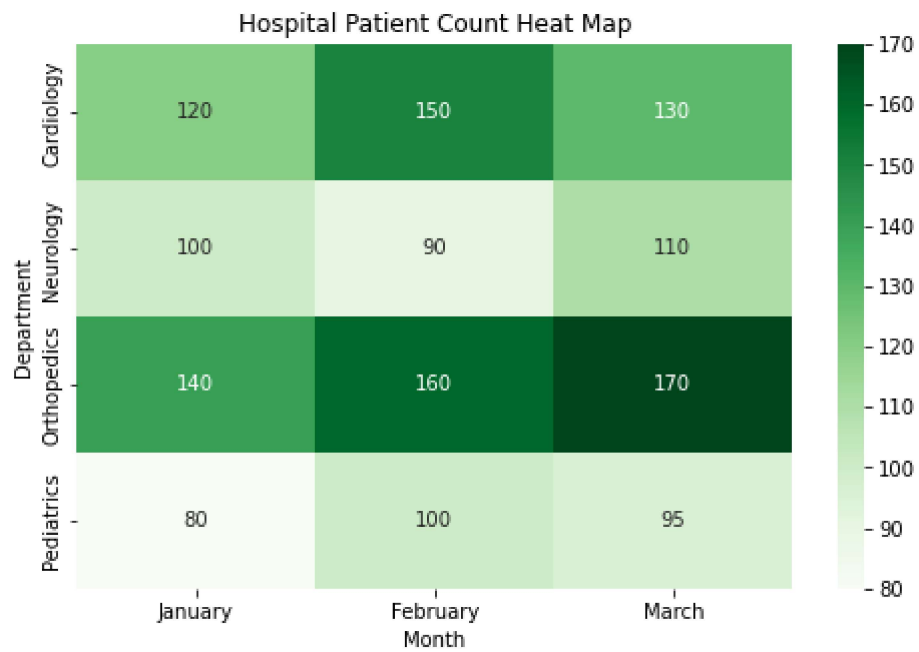
In [30]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = {
    'Department': ['Cardiology', 'Neurology', 'Orthopedics', 'Pediatrics'],
    'January': [120, 100, 140, 80],
    'February': [150, 90, 160, 100],
    'March': [130, 110, 170, 95]
}
df = pd.DataFrame(data)
df.set_index('Department', inplace=True)
print("Department :")
print(df)
print("\n")
plt.figure(figsize=(8, 5))
sns.heatmap(df, annot=True, cmap='Greens', fmt='d')
plt.title('Hospital Patient Count Heat Map')
plt.xlabel('Month')
plt.ylabel('Department')
plt.show()
Department_totals = df.sum(axis=1)
Month_totals = df.sum(axis=0)
print("Total Number of patients by departments:")
print(Department_totals)
print("\nTotal number of patients by Month:")
print(Month_totals)
print("\n")
max_Department = Department_totals.idxmax()
min_Month = Month_totals.idxmin()
print(f"Department with highest number of patients: {max_Department} ({Department_totals[max_Department]})")
print(f"Month with lowest number of patients: {min_Month} ({Month_totals[min_Month]})")

```

```

Department :
      January  February  March
Department
Cardiology    120      150   130
Neurology     100       90   110
Orthopedics   140      160   170
Pediatrics     80      100    95

```



Total Number of patients by departments:

```
Department
Cardiology    400
Neurology     300
Orthopedics   470
Pediatrics    275
dtype: int64
```

Total number of patients by Month:

```
January    440
February   500
March      505
dtype: int64
```

Department with highest number of patients: Orthopedics (470)

Month with lowest number of patients: January (440)